

AAP — AUTONOMOUS AGENT PLATFORM

Arquitectura Alfa 1 — Documento de diseño y manual de operación

Versión del documento: 1.0

Fecha: 30 de agosto de 2026

Estado del proyecto: diseño. Cero líneas de código escritas. Carpeta de proyecto vacía.

Autor del diseño: sesión de arquitectura (Principal Systems Architect / AI Agent Architect)

Destinatario: el operador del proyecto y, posteriormente, el agente de coding que implementará.

Índice

PARTE 0 — MANUAL: DÓNDE ESTÁS PARADO

- 0.1 Qué es esto en una frase
- 0.2 Las tres cosas que tienes que entender antes de nada
- 0.3 Estado real hoy
- 0.4 Cómo leer el resto del documento
- 0.5 Los tres primeros pasos concretos (los harás sin GPU)
- 0.6 Advertencia principal, antes de empezar

PARTE I — DISEÑO

- 1. QUÉ ESTAMOS CONSTRUYENDO REALMENTE
 - 1.1 Formulación honesta
 - 1.2 Lo que NO estamos construyendo
 - 1.3 La cadena de valor que la arquitectura debe soportar
- 2. QUÉ PROBLEMA TÉCNICO ESTAMOS RESOLVIENDO REALMENTE
- 3. MODELO MENTAL: QUÉ ES UN AGENTE EN ESTE SISTEMA
 - 3.1 Definición formal
 - 3.2 La distinción que más importa: las tres entidades
 - 3.3 Metáfora operativa: el trabajador
- 4. PRINCIPIOS ARQUITECTÓNICOS
- 5. ARQUITECTURA PROPUESTA
 - 5.1 Vista de capas
 - 5.2 Vista de flujo de una ejecución
- 6. RESPONSABILIDAD DE CADA COMPONENTE
 - 6.1 Agent Runtime
 - 6.2 Agent Definition
 - 6.3 Agent Instance / Run
 - 6.4 Visual Builder
 - 6.5 LLM Interface
 - 6.6 Model Router
 - 6.7 Tool Broker
 - 6.8 Policy Engine
 - 6.9 Memory & State Services
 - 6.10 Event Log
 - 6.11 Evaluation Engine
 - 6.12 Control Plane (FastAPI)
 - 6.13 Agent Factory
- 7. SEPARACIÓN RUNTIME / AGENT / UI / FACTORY
- 8. EL CICLO AUTÓNOMO Y LOS NIVELES DE AUTONOMÍA
 - 8.1 El ciclo canónico
 - 8.2 La corrección importante: no todos los agentes deben ejecutar el ciclo completo
 - 8.3 Consecuencia de diseño: la composición vence a la autonomía
 - 8.4 Criterios de terminación (obligatorios en todos los niveles)
- 9. MEMORIA, ESTADO, EVENTOS Y CONOCIMIENTO
 - 9.1 Las seis categorías, separadas por la fuerza
 - 9.2 Las decisiones y por qué
 - 9.3 Sobre embeddings y bases vectoriales

- 9.4 Eventos: qué se registra realmente
- 10. TOOL SYSTEM
 - 10.1 Contrato de una Tool
 - 10.2 Catálogo inicial, con criterio
 - 10.3 Cómo se registran las tools
- 11. POLICY ENGINE Y SEGURIDAD
 - 11.1 Por qué es estructural y no una capa de validación
 - 11.2 Modelo de política
 - 11.3 El presupuesto es una política, no una métrica
 - 11.4 Aprobación humana
 - 11.5 Secretos
- 12. EVALUATION
 - 12.1 El problema que nadie admite
 - 12.2 Las tres capas
 - 12.3 Los indicadores mínimos por agente
 - 12.4 Comparación de versiones
- 13. AUTO-MEJORA: QUÉ AUTOMATIZAR Y QUÉ NO
 - 13.1 Postura
 - 13.2 El bucle real
 - 13.3 Qué sí puede automatizarse desde el principio
 - 13.4 Qué requiere humano
- 14. AGENT DEFINITION: EL FORMATO Y LA FUENTE DE VERDAD
 - 14.1 La decisión de formato (y por qué esta y no otra)
 - 14.2 El schema (v1)
 - 14.3 Propiedades que este schema garantiza
- 15. VISUAL BUILDER: EL NO-CODE COMO PROPIEDAD ARQUITECTÓNICA
 - 15.1 La corrección de criterio más importante de esta sección
 - 15.2 Las pantallas y sus controles
 - 15.3 Cómo se evita que el modo avanzado contamine el no-code
 - 15.4 Cómo funcionaría técnicamente el canvas (etapa B2)
- 16. AGENT FACTORY, DUPLICACIÓN Y VERSIONADO
 - 16.1 Qué es runtime y qué es factory
 - 16.2 Plantillas
 - 16.3 Duplicación
 - 16.4 Versionado: la decisión
- 17. LLM INTERFACE Y MODEL ROUTING
 - 17.1 El contrato
 - 17.2 Por qué la cadena de degradación es una pieza de arquitectura y no un detalle
 - 17.3 Routing: contrato ahora, inteligencia después
 - 17.4 La regla que ahorra más dinero de todas
- 18. CLOUD VS LOCAL: LA FRONTERA OPERACIONAL
 - 18.1 La corrección: son tres capas, no dos
 - 18.2 Qué va dónde, exhaustivo
 - 18.3 Cuándo encender la GPU (criterio económico)
- 19. DOCKER: ARQUITECTURA DE EJECUCIÓN
 - 19.1 Principio
 - 19.2 Qué va dentro y qué va fuera
 - 19.3 La propiedad de reconstrucción (requisito explícito)
- 20. ESTRUCTURA DEL REPOSITORIO

- 20.1 Propuesta, con justificación de lo que se elimina
- 20.2 Las tres reglas de dependencia (verificables en CI)
- 21. MODELO DE DATOS (SQLite)
 - 21.1 Tres bases de datos, no una
 - 21.2 Esquema mínimo
 - 21.3 Nota sobre el crecimiento y la migración
- 22. API
 - 22.1 Qué es FastAPI aquí
 - 22.2 Endpoints
 - 22.3 Tres decisiones de contrato
- 23. OPTIMIZACIÓN DE COSTE
- 24. ARQUITECTURA MÍNIMA VIABLE (V1)
 - 24.1 Qué entra
 - 24.2 Qué queda explícitamente fuera de la V1, y por qué
 - 24.3 Criterio de "V1 terminada"

PARTE III — IMPLEMENTACIÓN

- 25. PROTOCOLO DE SESIÓN DE TRABAJO
- 26. HOJA DE RUTA POR HITOS
- 27. LO QUE YA SE HA HECHO Y LO QUE FALTA
- 28. CÓMO ENTREGAR ESTO A UN AGENTE DE CODING

PARTE IV — RIESGOS Y CUESTIONES ABIERTAS

- 29. RIESGOS
- 30. PREGUNTAS ARQUITECTÓNICAS AÚN ABIERTAS
- 31. GLOSARIO
- 32. CORRESPONDENCIA CON EL BRIEF ORIGINAL
- 33. CIERRE: LAS CINCO COSAS QUE NO HAY QUE ROMPER

PARTE 0 — MANUAL: DÓNDE ESTÁS PARADO

Esta parte existe para que en cualquier momento —dentro de tres días o dentro de tres meses, con o sin GPU encendida— puedas abrir este documento y reconstruir en cinco minutos el mapa completo de dónde está el proyecto, qué significa cada pieza y cuál es el siguiente paso concreto.

0.1 Qué es esto en una frase

Estás construyendo un **runtime pequeño que ejecuta agentes definidos por datos, no por código**, y una capa de fabricación encima que permite crear, duplicar, versionar y evaluar esos agentes desde una interfaz visual, de modo que añadir el agente número cincuenta no requiera tocar el motor. Todo lo demás —el scraping, WhatsApp, Android, los leads, los modelos Qwen, la GPU de Vast.ai— son **capacidades enchufables o combustible**, no el producto.

0.2 Las tres cosas que tienes que entender antes de nada

- 1. La Definición del Agente es el centro del universo.** No el código. No la interfaz. No el modelo. Un agente es un documento JSON validado por schema. La UI lo edita. La API lo edita. El runtime lo ejecuta. Git lo versiona. Si esa idea se respeta con disciplina, el no-code sale gratis y el versionado sale gratis. Si se rompe una sola vez —si un agente necesita "un pequeño if en el runtime"— todo el proyecto se degrada a un monolito con agentes hardcodeados y una UI decorativa.
- 2. El modelo propone, el runtime dispone.** El LLM nunca ejecuta nada. Produce una *intención* ("quiero llamar a la herramienta http.get con esta URL"). Esa intención atraviesa un punto de control obligatorio —el Policy Engine— que decide si se permite, se deniega o requiere aprobación humana, y contabiliza el presupuesto. No hay ninguna ruta alternativa hacia la ejecución. Este es el componente que impide que el sistema se destruya a sí mismo o queme 400 dólares de GPU en un bucle.
- 3. La GPU es combustible; el estado es el proyecto.** Todo lo que importa —definiciones, estado, memoria, historial, evaluaciones— vive fuera de la GPU y sobrevive a su destrucción. Una instancia cloud es un motor de inferencia desechable que se alcanza por una URL. Si apagar la GPU rompe algo que no sea "no puedo pensar ahora mismo", hay un error de arquitectura.

0.3 Estado real hoy

Elemento	Estado
Carpeta Arquitectura alfa 1	Vacía. Este documento es el primer contenido.
Repositorio Git	No creado.
Runtime	No existe.
Agent Definition Schema	Especificado en este documento, no implementado.
Base de datos	Diseñada (SQLite, 14 tablas), no creada.
UI	Especificada a nivel de pantallas y controles, no implementada.
Trabajos previos (Demand Hunter, embeddings, dedup, schedulers, ADB)	Existen como experiencia y posiblemente código externo. Se tratarán como fuente de herramientas y de un agente vertical, no como base del runtime.

0.4 Cómo leer el resto del documento

- **Parte I (Diseño):** la arquitectura. Es lo que se entrega al agente de coding. Léela una vez entera, aunque haya partes que no uses hasta el mes tres.
- **Parte II (Especificaciones):** schemas, tablas, endpoints, estructura de repo. Es material de referencia; se consulta, no se lee.
- **Parte III (Implementación):** el plan por hitos y el protocolo de sesión de GPU. Es lo que abres cada vez que te sientas a construir.
- **Parte IV (Riesgos y preguntas abiertas):** lo que puede matar el proyecto y lo que aún no está decidido.

0.5 Los tres primeros pasos concretos (los harás sin GPU)

1. **Crear el repositorio y el esqueleto** (Hito M0). No requiere GPU, no requiere modelo. Una hora de trabajo. Sin esto, nada más es posible.
2. **Implementar el Agent Definition Schema y el registro en SQLite** (M1). Tampoco requiere GPU. Es la pieza más importante del sistema entero.
3. **Implementar el LLM Interface con un provider falso (echo)** (M2). Permite ejecutar un agente completo de principio a fin sin gastar un céntimo de inferencia, y hace que todo el resto del sistema sea testeable sin GPU.

Solo a partir del hito M6 aparece una razón real para encender una GPU. **Antes de ese punto, alquilar GPU es quemar dinero.** Esta es la primera corrección de criterio importante de este documento y se justifica en la sección 15.

0.6 Advertencia principal, antes de empezar

El brief que originó este documento pide una fábrica de trabajadores digitales. La fábrica es un objetivo correcto y la arquitectura de este documento la soporta. Pero el riesgo dominante del proyecto **no es técnico, es de secuencia**: construir la fábrica antes de tener un solo trabajador que produzca un resultado económico verificable.

La regla de gobierno que propongo, y que atraviesa todo el diseño:

La fábrica se construye como subproducto obligado de fabricar el primer agente vertical, no antes de él.

Concretamente: a partir del hito M6, todo lo que se construya debe estar justificado por una necesidad real del primer agente (Demand Hunter), y toda pieza que ese agente necesite debe construirse de forma genérica en el núcleo. Nada específico de un agente entra en `core/`. Nada genérico se queda fuera de `core/`. Esa tensión, mantenida con disciplina durante dos o tres agentes, es la fábrica.

Si en cambio se construyen doce meses de plataforma abstracta sin un agente que genere una sola oportunidad real, el resultado será una infraestructura elegante que nadie usa y cuyas abstracciones estarán mal, porque no habrán sido corregidas por ningún contacto con la realidad.

PARTE I — DISEÑO

1. QUÉ ESTAMOS CONSTRUYENDO REALMENTE

1.1 Formulación honesta

Estamos construyendo **un intérprete de agentes**.

De la misma forma que un intérprete de Python ejecuta programas descritos en un lenguaje sin conocer nada de esos programas, el AAP ejecuta *agentes* descritos en un lenguaje declarativo sin conocer nada de esos agentes. El "lenguaje" es la Agent Definition. El "intérprete" es el Agent Runtime. La "IDE" es el Visual Builder. El "sistema operativo" que le da capacidades y le pone límites son el Tool System y el Policy Engine.

Esta analogía no es decorativa: es el criterio de diseño. Cada vez que dudes sobre dónde va una funcionalidad, pregunta *"¿esto sería una función del intérprete de Python, o sería código de un programa escrito en Python?"*. Buscar leads es código de programa. Reintentar una llamada fallida es función del intérprete.

1.2 Lo que NO estamos construyendo

Es igual de importante y hay que fijarlo por escrito:

- **No** un chatbot ni un producto conversacional.
- **No** un framework de agentes de propósito general que compita con LangGraph, CrewAI o similares. Sería una guerra perdida y además irrelevante para el objetivo económico.
- **No** una herramienta de scraping. El scraping es *una tool* entre veinte.
- **No** un orquestador de workflows tipo n8n. Aunque compartirá un canvas visual, la unidad ejecutable es un agente con objetivo, no un flujo de pasos.
- **No**, todavía, un producto SaaS multi-tenant. Un solo operador, una sola instalación, cero gestión de cuentas y de facturación en la V1. Añadir multi-tenancy después cuesta trabajo pero es viable; anticiparlo ahora mata la velocidad.

1.3 La cadena de valor que la arquitectura debe soportar

El brief plantea la progresión `DATA → SIGNAL → OPPORTUNITY → ACTION → OUTCOME`. Es correcta y merece traducción arquitectónica explícita, porque cada eslabón exige un componente distinto:

Eslabón	Qué es	Qué componente lo soporta
DATA	Hechos crudos capturados del mundo	Tools de ingesta + Entity Store
SIGNAL	Un hecho que supera un umbral de relevancia	Reglas o modelo evaluando entidades; se persiste como entidad tipada
OPPORTUNITY	Una señal a la que se le asigna un valor esperado y un dueño	Entidad de dominio + scoring; nunca "en la cabeza del LLM"
ACTION	Un acto sobre el mundo exterior	Tool call autorizada por Policy Engine
OUTCOME	La consecuencia medible de la acción, que llega tarde	Tabla de outcomes con atribución a <code>run_id</code> y <code>agent_version_id</code>

La observación crítica —y la que casi todo el mundo omite— es que OUTCOME llega días o semanas después de la ACTION. Un lead contactado hoy responde el jueves y firma en tres

semanas. Si la arquitectura no guarda desde el minuto uno la trazabilidad `outcome → run → versión de agente`, jamás podrás responder a la única pregunta que importa: *¿la versión 2 del agente gana más dinero que la versión 1?*. Esto obliga a que los outcomes sean una entidad de primera clase con escritura diferida y asíncrona, no un campo del log de la ejecución. Está recogido en el modelo de datos (§17) y en la evaluación (§11).

2. QUÉ PROBLEMA TÉCNICO ESTAMOS RESOLVIENDO REALMENTE

Conviene nombrar el problema con precisión, porque determina qué es éxito.

Problema 1 — El acoplamiento entre lógica de agente y código de motor. La forma natural de escribir un agente es un script: un bucle, unos prompts, unas llamadas a APIs. Funciona para uno. Con cinco agentes tienes cinco scripts que divergen; con veinte tienes un pantano donde arreglar un bug de reintentos significa editarlo en veinte sitios y romper tres. La solución no es "mejor código", es **mover la lógica del agente de código a datos**.

Problema 2 — La ausencia de un chokepoint de autoridad. Un LLM con acceso a shell, red y disco es un ejecutor no determinista con permisos de administrador. El problema no es que "el modelo se vuelva malo": es que llamará a `rm` con una ruta mal formada, entrará en un bucle de reintentos que consumirá el presupuesto, o hará 4.000 peticiones a una API que te bloqueará la IP. Se resuelve con un único punto de paso obligatorio entre decisión y ejecución.

Problema 3 — La no-observabilidad de los procesos no deterministas. Cuando un agente falla, la pregunta "¿por qué?" es incontestable si no existe una traza estructurada de qué observó, qué decidió, con qué contexto y qué devolvió cada herramienta. Sin eso no hay depuración, no hay evaluación y no hay mejora: solo hay superstición sobre prompts.

Problema 4 — La volatilidad de la infraestructura de cómputo. El estado valioso y el cómputo caro tienen ciclos de vida opuestos. El diseño debe separarlos físicamente para que uno pueda morir sin arrastrar al otro.

Problema 5 — La distancia entre configurar y programar. Que una persona no técnica —o tú mismo un martes cansado— pueda crear un agente nuevo en diez minutos, sin abrir un editor, es la diferencia entre una plataforma y un repositorio de scripts. Pero es *consecuencia* de resolver bien el problema 1, no un problema independiente.

Si al final del proyecto estos cinco problemas están resueltos y solo existen tres agentes, el proyecto es un éxito. Si hay cuarenta agentes y estos problemas no están resueltos, es un fracaso caro.

3. MODELO MENTAL: QUÉ ES UN AGENTE EN ESTE SISTEMA

3.1 Definición formal

Un **agente** es una máquina de estados con objetivo, capacidad de percepción y de acción limitadas por política, memoria persistente y criterio de terminación, cuya función de transición está parcialmente delegada a un modelo de lenguaje.

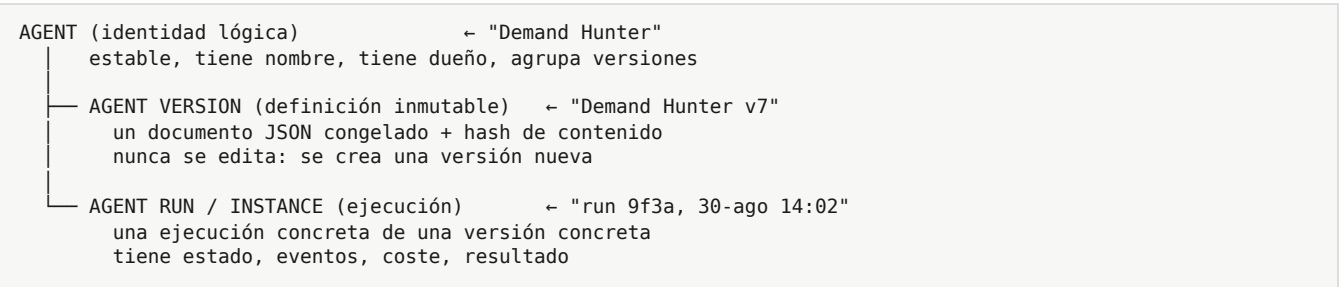
Desglose de cada término, porque cada uno tiene consecuencias:

- **Máquina de estados:** en todo momento un agente en ejecución está en un estado nombrable y persistido. No es "un proceso opaco pensando". Si el proceso muere, el estado permite reanudar o al menos diagnosticar.

- **Con objetivo:** existe un `goal` explícito, y una `success_criteria` verificable. Un agente sin criterio de éxito no es evaluable y por tanto no es mejorable.
- **Percepción y acción limitadas por política:** sus sentidos y sus manos son exactamente las tools que su definición declara y su política autoriza. Nada más. No hay acceso ambiental implícito.
- **Memoria persistente:** sobrevive a la ejecución. Ver §8 para la separación estricta entre las cinco cosas que la gente llama "memoria".
- **Criterio de terminación:** todo agente termina. Por éxito, por fallo, por agotamiento de presupuesto o por intervención. No existe "corre para siempre"; existe "se ejecuta periódicamente".
- **Función de transición parcialmente delegada al LLM:** la parte no determinista está acotada. El LLM elige *qué hacer a continuación* dentro de un espacio de opciones que el runtime define; no controla el bucle.

3.2 La distinción que más importa: las tres entidades

Este es el punto donde el brief pide rigor y donde la mayoría de plataformas fracasan. Son tres cosas completamente distintas, con ciclos de vida distintos y almacenamiento distinto:



Reglas derivadas, no negociables:

1. **Las versiones son inmutables.** Editar en la UI no modifica la versión activa: crea un borrador (`draft`) que al guardarse se convierte en una versión nueva. Esto hace el rollback trivial y la evaluación comparable.
2. **Un run apunta siempre a una versión, nunca a un agente.** Sin esto la evaluación es imposible.
3. **La configuración de despliegue no vive en la versión.** Qué modelo físico se usa, qué endpoint, qué credenciales, en qué máquina corre: eso es *entorno*, y cambia sin crear versión nueva. La versión declara *necesidades* ("necesito razonamiento pesado"), el entorno declara *recursos* ("razonamiento pesado = Qwen3 en http://..."). Confundirlos genera versiones no reproducibles.

3.3 Metáfora operativa: el trabajador

Para pensar el producto, no la implementación:

Concepto humano	Concepto del sistema
Puesto de trabajo (descripción del rol)	Agent Definition
El empleado concreto en su turno del martes	Agent Run
Su formación y manuales	Knowledge
Lo que recuerda de clientes anteriores	Long-term Memory / Entity Store
Lo que tiene en la cabeza ahora mismo	Working Memory (efímera)
Sus herramientas y accesos	Tools + credenciales
El reglamento de la empresa	Policies
Su evaluación de desempeño	Evaluation

Concepto humano	Concepto del sistema
Un ascenso o cambio de funciones	Nueva versión

La metáfora es útil hasta un punto y luego engaña: un empleado aprende solo, un agente no. El aprendizaje aquí es un cambio de configuración explícito, propuesto por el sistema y aprobado por un humano (§12).

4. PRINCIPIOS ARQUITECTÓNICOS

Estas son las reglas de gobierno. Están ordenadas por prioridad: cuando dos entren en conflicto, gana la de arriba.

P1 — Definition-first. Todo comportamiento configurable de un agente vive en su Definición. Si algo no se puede expresar en la Definición, o bien se extiende el schema, o bien es una tool nueva. Jamás una rama `if agent.name == "..."` en el runtime.

P2 — El modelo no tiene autoridad. Ninguna acción sobre el mundo ocurre sin pasar por el Policy Engine. Ni una. El punto de paso es estructural (imposible de saltar por construcción), no una convención.

P3 — Todo lo que ocurre se registra como evento. Un run es reconstruible desde su log de eventos. El log es apéndice-only y es a la vez traza, auditoría y materia prima de la evaluación. No hay tres subsistemas de observabilidad: hay uno.

P4 — El estado sobrevive al cómputo. Ninguna pieza de información valiosa vive solo en RAM, solo en la GPU o solo en el contexto de un LLM.

P5 — Todo run tiene presupuesto. Tokens, dinero, tiempo de reloj, número de pasos y número de llamadas a herramienta. Superar el presupuesto termina el run de forma limpia y registrada. Un agente sin presupuesto es una fuga financiera con permisos.

P6 — La UI y la API son iguales ante el sistema. Ambas son clientes del mismo control plane. No existe funcionalidad accesible solo por UI. Esto garantiza que el no-code no sea una capa mentirosa.

P7 — Una sola máquina hasta que se demuestre lo contrario. Un proceso, SQLite, sin colas distribuidas, sin Kubernetes, sin Kafka. Cada componente distribuido debe pagar su entrada con un problema medido, no anticipado.

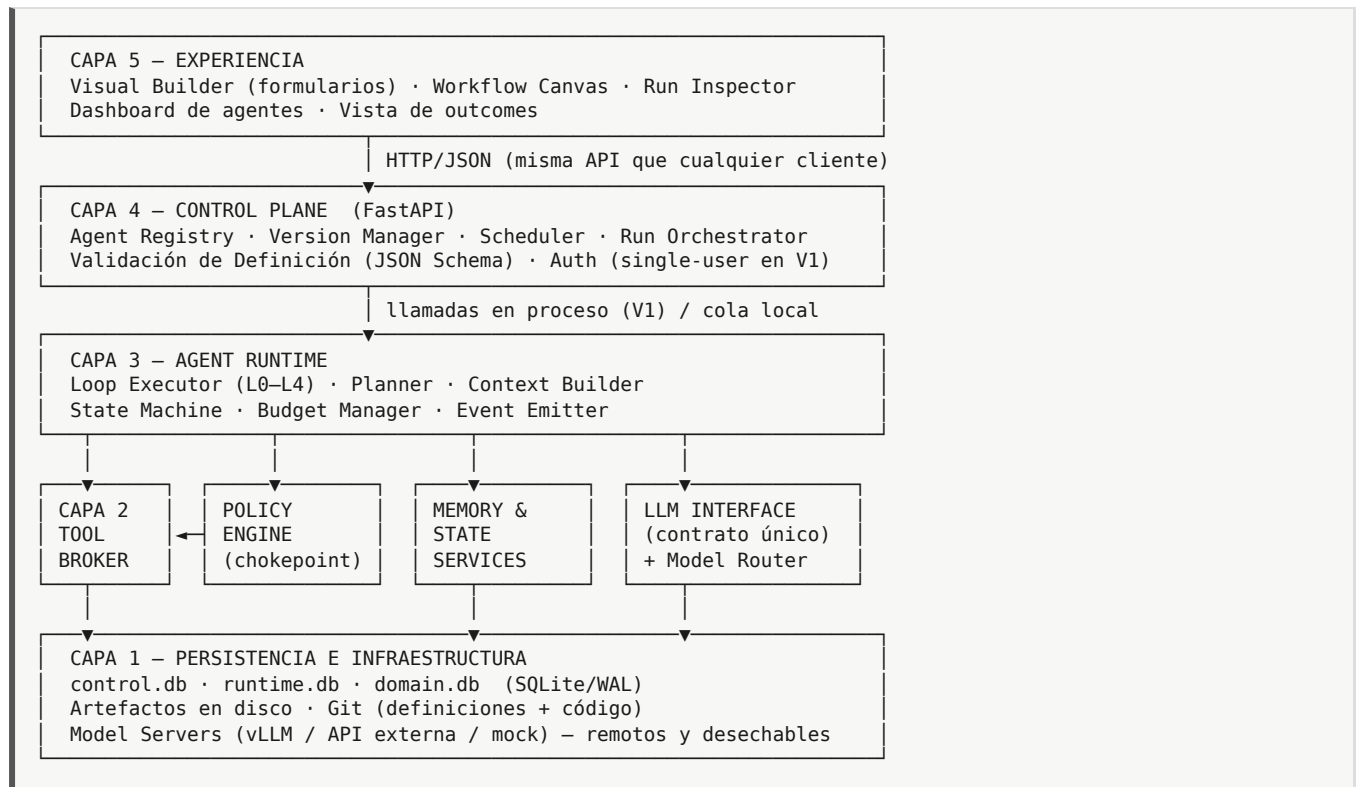
P8 — Abstracción a la segunda repetición, no a la primera. Cuando aparece el segundo caso se generaliza. Con un solo caso, se escribe concreto. La sobre-abstracción prematura es el modo de fallo más probable de este proyecto concreto, dado el nivel de ambición del brief.

P9 — Fallar de forma ruidosa y barata. Timeouts en todo, reintentos con límite explícito, errores tipados. Un agente que se cuelga silenciosamente es peor que uno que falla.

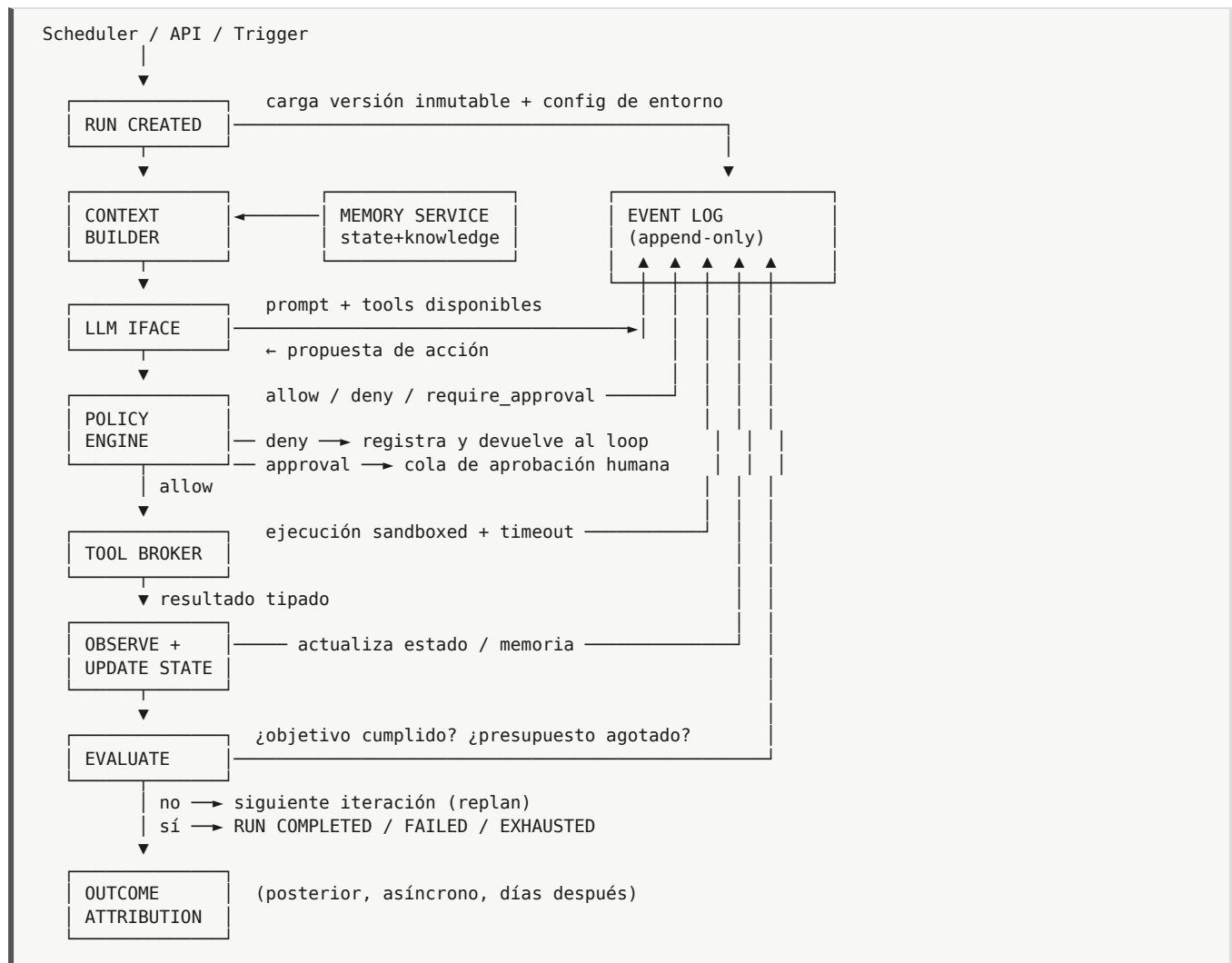
P10 — Reconstruibilidad total. `git clone` + `docker compose up` + restaurar snapshot de estado = sistema operativo. Cualquier paso manual no documentado es un defecto.

5. ARQUITECTURA PROPUESTA

5.1 Vista de capas



5.2 Vista de flujo de una ejecución



6. RESPONSABILIDAD DE CADA COMPONENTE

El formato es deliberado: *qué hace* y sobre todo *qué NO hace*. La mitad de la arquitectura es la segunda columna.

6.1 Agent Runtime

- **Hace:** carga una versión, construye contexto, ejecuta el bucle correspondiente al nivel de autonomía, mantiene la máquina de estados, aplica presupuestos, emite eventos, decide terminación.
- **No hace:** no conoce ningún agente concreto. No sabe qué es un lead. No habla HTTP con el mundo exterior. No decide si una acción es permitida. No formatea prompts específicos de dominio (eso viene de la Definición).

6.2 Agent Definition

- **Hace:** describe *qué* es el agente y *qué* puede hacer, de forma declarativa, completa y validable.
- **No hace:** no contiene código. No contiene credenciales. No contiene endpoints físicos ni nombres de modelos concretos. No contiene estado.

6.3 Agent Instance / Run

- **Hace:** representa una ejecución con su estado, su presupuesto consumido, sus eventos y su resultado.
- **No hace:** no puede modificar su propia definición durante la ejecución. (Excepción controlada: puede *proponer* un cambio, que va a la cola de mejoras. §12.)

6.4 Visual Builder

- **Hace:** leer y escribir Definiciones a través de la API, validarlas contra el schema antes de guardar, mostrar el estado de los runs y los resultados.
- **No hace:** no genera código. No contiene lógica de negocio. No es el único camino para hacer nada.

6.5 LLM Interface

- **Hace:** ofrece un contrato único de inferencia (`complete`), traduce al dialecto de cada provider, cuenta tokens y coste, aplica timeouts y reintentos, normaliza la salida de tool-calling entre modelos que la implementan de forma distinta.
- **No hace:** no decide qué modelo usar (eso es el Router). No mantiene conversación ni historial. No interpreta el contenido semántico de las respuestas.

6.6 Model Router

- **Hace:** traduce una *clase de capacidad* requerida (`cheap` / `standard` / `heavy` / `coding` / `embedding`) más señales de la tarea (longitud, criticidad, latencia tolerada) en un provider físico concreto, según una tabla de configuración del entorno. Degrada con elegancia si un provider no está disponible.
- **No hace:** no aprende, no optimiza automáticamente, no es un sistema de ML. En V1 es un diccionario y tres reglas. Esto es intencional.

6.7 Tool Broker

- **Hace:** mantiene el registro de tools, valida inputs contra el JSON Schema declarado, ejecuta con timeout y aislamiento, normaliza errores, mide latencia y emite eventos.
- **No hace:** no autoriza (pregunta al Policy Engine). No conoce el agente que la invoca más allá de su contexto de seguridad.

6.8 Policy Engine

- **Hace:** dada la terna (contexto del agente, tool, argumentos), devuelve `ALLOW` / `DENY(motivo)` / `REQUIRE_APPROVAL` . Verifica permisos declarados, restricciones de argumentos (rutas, dominios, tablas), y presupuesto restante.
- **No hace:** no ejecuta nada. No es configurable por el propio agente en tiempo de ejecución. No tiene modo "desactivado".

6.9 Memory & State Services

- **Hace:** persistir y recuperar las cinco categorías separadas de §8, con APIs distintas para cada una.
- **No hace:** no es un cajón desastre. No guarda automáticamente todo lo que pasa "por si acaso".

6.10 Event Log

- **Hace:** registro append-only, tipado, ordenado, consultable, de todo lo relevante que ocurre en un run.
- **No hace:** no es un bus de mensajes. No es el mecanismo de control de flujo. Nadie "espera" a un evento en V1.

6.11 Evaluation Engine

- **Hace:** calcula métricas mecánicas siempre; ejecuta evaluación por rúbrica bajo demanda; agrega outcomes externos; compara versiones.
- **No hace:** no promueve versiones automáticamente (salvo en el caso acotado de §13.4). No inventa ground truth.

6.12 Control Plane (FastAPI)

- **Hace:** CRUD de agentes y versiones, disparo y control de runs, consulta de estado y trazas, cola de aprobaciones, scheduling.
- **No hace:** no ejecuta el bucle del agente en el hilo de la petición HTTP. Los runs son asíncronos desde el primer día; esto no es sobre-ingeniería, es evitar reescribir todo en el mes dos.

6.13 Agent Factory

- **Hace:** plantillas, clonado, wizard de creación, diffs entre versiones, promoción y archivado.
- **No hace:** no es un componente de ejecución. Es una capa sobre el Control Plane. **No requiere ningún código de runtime.** Si la Factory necesitara tocar el runtime, P1 se habría roto.

7. SEPARACIÓN RUNTIME / AGENT / UI / FACTORY

Cuatro cuadrantes con frecuencias de cambio muy distintas. Esa es exactamente la razón de separarlos:

	CAMBIA POCO	CAMBIA MUCHO
CÓDIGO	RUNTIME loop, policy, broker ~5.000 líneas cambia mensualmente	TOOLS integraciones concretas crecen sin límite cambian semanalmente
DATOS	SCHEMA de definición cambia trimestralmente (con migración)	AGENT DEFINITIONS cambian a diario (sin tocar código)

Criterio de frontera, aplicable en caliente: si para añadir un agente nuevo tienes que escribir Python, la frontera está mal trazada — *excepto* si lo que necesitas es una capacidad nueva del mundo real (una tool). Ese es el único código legítimo que un agente nuevo puede exigir.

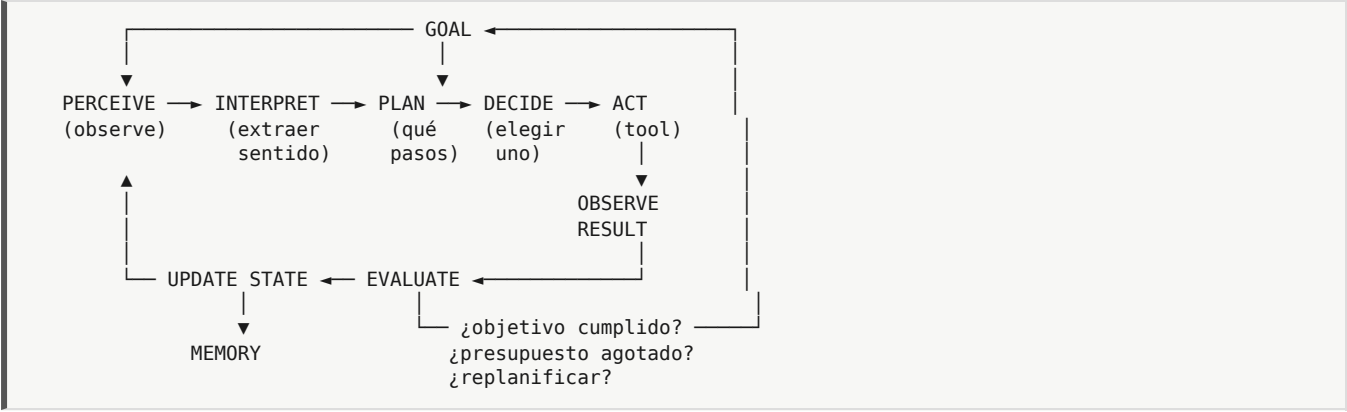
Prueba de fuego de la arquitectura, que hay que ejecutar literalmente en el hito M9:

Crear "Lead Discovery Agent" duplicando "Demand Hunter", cambiando objetivo, dos tools, la política de red y el criterio de éxito, **sin un solo commit en core/** .

Si esa prueba no pasa, no se avanza al hito siguiente: se corrige el schema.

8. EL CICLO AUTÓNOMO Y LOS NIVELES DE AUTONOMÍA

8.1 El ciclo canónico



8.2 La corrección importante: no todos los agentes deben ejecutar el ciclo completo

El brief ya lo intuye al pedir niveles de autonomía, y es la intuición correcta. Voy más lejos: **la mayoría del valor económico de los agentes que vas a construir se producirá en los niveles 0 a 2.** Los niveles 3 y 4 son más caros, más lentos, mucho menos fiables y necesarios solo en una minoría de tareas. Diseñar todo el sistema para el nivel 4 y luego usarlo en nivel 1 es la forma habitual de acabar pagando 30 veces más por resultados peores.

Nivel	Nombre	Bucle	El LLM decide...	Coste típico por run	Cuándo usarlo
L0	Determinista	Sin bucle. Secuencia fija de tools.	Nada. Puede no haber LLM.	~0	Ingesta, ETL, sincronizaciones, envíos programados.
L1	Reactivo	Un paso: entrada → LLM → (0..n tools) → salida.	Cómo transformar/clasificar/redactar.	1 llamada	Clasificar señales, extraer campos, redactar un mensaje, responder un WhatsApp.
L2	Planificado	Plan explícito al inicio, luego ejecución de pasos sin replanificar.	El plan, una vez.	1 + n llamadas	Investigación acotada, generación de propuesta, enriquecimiento de un lead.
L3	Iterativo	Bucle observar→decidir→actuar con replanificación, límite duro de iteraciones.	Qué hacer en cada paso.	n × llamadas	Búsqueda exploratoria, resolución de incidencias, tareas con incertidumbre real.

Nivel	Nombre	Bucle	El LLM decide...	Coste típico por run	Cuándo usarlo
L4	Autónomo	L3 + auto-generación de subobjetivos + memoria entre ejecuciones + criterio de parada propio.	Incluso qué objetivos perseguir.	Alto e impredecible	Casi nunca en V1. Requiere presupuestos estrictos y supervisión.

El nivel es un campo de la Definición del agente (`runtime.autonomy_level`), y el runtime implementa cinco ejecutores que comparten toda la infraestructura (contexto, policy, tools, eventos, presupuesto) y difieren solo en la forma del bucle. Son cinco funciones, no cinco sistemas.

8.3 Consecuencia de diseño: la composición vence a la autonomía

Un agente L3 caro que "investiga leads" suele ser derrotado en coste, latencia y fiabilidad por un pipeline de tres agentes L0/L1/L1 encadenados. **La arquitectura debe hacer barato encadenar agentes**, porque la composición de agentes simples es casi siempre superior a un agente complejo. De ahí que:

- Un agente pueda ser invocado como una tool por otro agente (`tool: agent.invoke`), con presupuesto propio y profundidad máxima de anidamiento (2 en V1, para prevenir recursión infinita).
- El workflow canvas exista: es precisamente el mecanismo de composición determinista de piezas no deterministas pequeñas.

8.4 Criterios de terminación (obligatorios en todos los niveles)

Todo run termina por exactamente una de estas causas, que se registra:

`COMPLETED` (criterio de éxito satisfecho) · `FAILED` (error irrecuperable) · `EXHAUSTED` (presupuesto agotado: pasos, tokens, dinero o tiempo) · `BLOCKED` (esperando aprobación humana más allá del timeout) · `CANCELLED` (intervención) · `CRASHED` (excepción no controlada; es un bug, y debe tener alerta).

9. MEMORIA, ESTADO, EVENTOS Y CONOCIMIENTO

Aquí está, en mi opinión, el punto donde más proyectos de agentes se rompen sin darse cuenta. La palabra "memoria" se usa para seis cosas distintas y al mezclarlas se acaba con un almacén vectorial gigante que envenena el contexto y del que nadie se fía.

9.1 Las seis categorías, separadas por la fuerza

#	Categoría	Vida	Volumen	Escritura	Almacenamiento V1	Entra en el prompt
1	Working Memory	Un run	Pequeño	Automática	RAM + reconstruible del event log	Sí, completo
2	Run State	Un run (persistido)	Pequeño	Explícita, tipada	<code>runtime.db:run_state</code> (JSON)	Parcial, seleccionado
3	Event History	Permanente	Grande	Automática, append-only	<code>runtime.db:events</code>	No (salvo resumen)
4	Long-term Memory	Permanente	Medio	Explícita y curada	<code>control.db:memories</code> (+ embeddings opcionales)	Sí, top-k recuperado

#	Categoría	Vida	Volumen	Escritura	Almacenamiento V1	Entra en el prompt
5	Knowledge	Ligada a la versión	Medio	Humana / import	Ficheros + índice; referenciado por la Definición	Sí, recuperado
6	Entity Store (dominio)	Permanente	Muy grande	Vía tools	domain.db (tablas de negocio)	No: se consulta con tools

9.2 Las decisiones y por qué

Working Memory no se persiste como tal. Es el contexto de trabajo del run actual. Persistirla es duplicar el event log. Si un run se reanuda, la working memory se *reconstruye* proyectando sus eventos. Esto elimina un subsistema entero.

Run State es explícito y tipado, no libre. La Definición declara `state_schema` (un objeto JSON pequeño: p. ej. `{leads_encontrados: int, fase: str, ultimo_cursor: str}`). El agente solo puede escribir en esas claves. Un estado de forma libre se convierte en un vertedero en tres semanas y hace imposible detectar corrupción.

Los eventos son la verdad de lo que pasó, y son también los logs. Rechazo explícitamente tener un subsistema de eventos y otro de logging y otro de tracing. Una tabla, tipada, con nivel (`DEBUG|INFO|WARN|ERROR|AUDIT`). Se consulta por `run_id`. Sobre esa tabla se construyen el inspector de runs, la evaluación mecánica y la auditoría. Tres funciones, una fuente.

Long-term Memory se escribe de forma explícita, nunca automática. Esta es la decisión contraintuitiva y creo que la correcta. La tentación es "guardar todo lo aprendido". El resultado real es un almacén de afirmaciones no verificadas, contradictorias y sin fecha, que recuperado semánticamente contamina el razonamiento futuro con basura pasada. Reglas:

- Escribir en memoria de largo plazo es **una tool** (`memory.write`), sujeta a política, con schema: `{tipo, sujeto, afirmación, evidencia (run_id/fuente), confianza, caduca_en}`.
- Toda memoria tiene procedencia y caducidad. Sin `run_id` de origen no se admite.
- Existe un límite de memorias por agente (p. ej. 500 en V1). Al superarlo, se comprime o expira lo menos usado. La escasez impuesta obliga a que solo lo valioso sobreviva.
- La recuperación es top-k con filtro por tipo y frescura, no una búsqueda semántica global.

Knowledge es de solo lectura y pertenece a la versión. Documentos, guías, ejemplos, listas de precios. Cambiar el knowledge **debe crear una versión nueva del agente**, porque cambia su comportamiento tanto como cambiar el prompt. Esta es una de las decisiones que más limpia hace la evaluación: si el conocimiento pudiera cambiar bajo los pies del agente, comparar v1 con v2 no significaría nada.

El Entity Store es la pieza que el brief no nombra y que probablemente es la más valiosa. Para un Demand Hunter, lo que importa no es que "recuerde" conversaciones: es que exista una tabla `companies`, otra `signals`, otra `leads`, otra `contacts`, otra `outreach`, con deduplicación, claves naturales, historial y estado. Eso no es memoria de agente: es la base de datos del negocio, y los agentes son sus escritores y lectores mediante tools. Consecuencia arquitectónica importante:

Los datos de valor económico no viven dentro del agente. Viven en un almacén de dominio que sobrevive a todos los agentes, a todas sus versiones y a todos sus fallos.

Esto también resuelve el problema de la deduplicación semántica que ya has trabajado antes: es una propiedad del Entity Store (con embeddings + índice), no del agente. Un agente nuevo hereda un mundo ya poblado en lugar de empezar de cero.

9.3 Sobre embeddings y bases vectoriales

Postura: **no introducir una base vectorial dedicada en V1**. SQLite con una tabla de vectores y búsqueda por fuerza bruta (o `sqlite-vec` si se necesita) resuelve hasta cientos de miles de vectores con latencias de milisegundos en una sola máquina. Qdrant, Weaviate o pgvector entran cuando haya una medición que demuestre que hacen falta, no antes. El coste de esa decisión, si te equivocas, es una tarde de migración; el coste de la decisión contraria es un componente más que mantener, desplegar y respaldar desde el día uno.

9.4 Eventos: qué se registra realmente

El brief lista doce tipos de evento. Reduzco a los que ganan su sitio en V1 —cada evento cuesta escritura, esquema y ruido—, marcando los que difiero:

Núcleo V1 (11 tipos): `run.started` · `run.finished` (con causa de terminación) · `step.started` · `llm.called` (modelo, tokens in/out, latencia, coste) · `decision.made` (acción propuesta + razonamiento resumido) · `policy.evaluated` (solo si deniega o requiere aprobación) · `tool.called` · `tool.result` (ok/error, latencia, tamaño) · `state.updated` (diff) · `memory.written` · `error.raised`.

Diferidos (no aportan aún): `AgentCreated/AgentStarted` (son control plane, van a una tabla de auditoría distinta, no al log de runs) · `TaskReceived` (redundante con `run.started`) · `ObservationCreated` (es `tool.result` con otro nombre) · `EvaluationCompleted` (la evaluación es un registro en su propia tabla, no un evento del run).

Cuándo eventos y cuándo llamada directa: en V1, **todo el control de flujo es por llamada directa en proceso**. Los eventos son puramente un registro. No hay suscriptores, no hay bus, no hay reacción a eventos. Introducir pub/sub sería exactamente el tipo de sobre-ingeniería que el brief pide evitar, y añadiría depuración no determinista a un sistema que ya es no determinista por el LLM. Si más adelante hace falta reaccionar a eventos entre agentes (p. ej., "cuando aparezca un lead con score > 80, dispara el Proposal Agent"), la forma correcta y barata es un **trigger declarativo en la Definición** que el scheduler evalúa por consulta periódica sobre el Entity Store. Una cola real solo cuando haya latencias medidas inaceptables.

10. TOOL SYSTEM

10.1 Contrato de una Tool

```
tool:
  id: http.get                # namespace.verbo – estable, es una API pública
  version: 1                  # cambios incompatibles suben versión
  title: "Petición HTTP GET"
  description: >              # ESTE TEXTO LO LEE EL MODELO. Es prompt, no documentación.
    Descarga el contenido de una URL pública. Devuelve status, headers y body
    truncado a 200 KB. No sigue más de 3 redirecciones. No sirve para páginas
    que requieren JavaScript.
  input_schema: {json-schema} # validado ANTES de ejecutar
  output_schema: {json-schema} # validado DESPUÉS de ejecutar
  permissions:                # capacidades que exige; el Policy Engine las contrasta
    - network.http
  side_effects: read           # read | write | destructive → gobierna aprobaciones
  idempotent: true
  timeout_s: 30
  retry: {max: 2, backoff: exponential, on: [timeout, 5xx]}
  cost_hint: {money: 0, latency_ms: 800}
  redact: [headers.authorization] # qué no se persiste en el event log
```

Notas de diseño que importan:

- **description es la interfaz para el modelo.** La calidad de esa frase determina la tasa de acierto del agente más que casi cualquier otra cosa. Debe decir explícitamente qué *no* hace la tool; es la forma más eficaz de evitar mal uso.
- **side_effects es el campo que conecta tools y políticas.** Permite reglas como "cualquier tool destructive requiere aprobación" sin enumerarlas una por una.
- **Validación de salida obligatoria.** Una tool que devuelve algo fuera de su schema es un error de la tool, no un problema del agente. Esto evita que basura no estructurada llegue al contexto.
- **redact** existe porque el event log guardará argumentos y resultados, y ahí acabarían tokens y credenciales si nadie lo impide.

10.2 Catálogo inicial, con criterio

No todas las tools que el brief menciona deben existir el primer día. Propuesta:

Tool	¿V1?	Justificación
<code>http.get</code> / <code>http.post</code>	Sí	Es la puerta al mundo. Sin ella no hay agente útil.
<code>search.web</code>	Sí	Vía una API de búsqueda. Es el sensor primario del Demand Hunter.
<code>db.query</code> / <code>db.upsert</code> (dominio)	Sí	Acceso al Entity Store, con whitelist de tablas y operaciones.
<code>memory.write</code> / <code>memory.search</code>	Sí	Núcleo del ciclo.
<code>knowledge.search</code>	Sí	RAG sobre el knowledge de la versión.
<code>agent.invoke</code>	Sí	Composición. Es lo que hace la arquitectura escalar por composición y no por complejidad.
<code>fs.read</code> / <code>fs.write</code>	Sí , restringido	Solo dentro del workspace del run.
<code>llm.extract</code> (extracción estructurada)	Sí	Patrón tan repetido que merece ser tool en vez de código de agente.
<code>shell.exec</code>	No en V1	Riesgo altísimo, beneficio bajo para los agentes previstos. Entra en la fase de Coding Agent, y en contenedor efímero.
<code>browser.*</code> (navegador headless)	Fase 2	Necesario cuando <code>http.get</code> no baste (JS, login). Caro en recursos.
<code>android.adb</code> / AutoX	Fase 2-3	Solo cuando exista el WhatsApp Worker. Muy valiosa pero con riesgo operativo alto (bloqueos de cuenta).
<code>git.*</code>	Fase 3	Solo para Coding Agent.
<code>email.send</code> / <code>whatsapp.send</code>	Fase 2	Acciones con efecto exterior irreversible → <code>side_effects: write</code> , aprobación por defecto.

Criterio general: una tool entra cuando un agente real la necesita para completar un objetivo real. No se construye un catálogo por completitud.

10.3 Cómo se registran las tools

Tres orígenes, en orden de preferencia:

1. **Built-in:** función Python decorada con `@tool(...)`, registrada al arrancar. El schema se deriva de los type hints y el docstring.

2. **HTTP tool declarativa**: definida en YAML apuntando a una API externa (URL, método, mapeo de parámetros, autenticación por referencia a un secreto). **No requiere escribir código** — este es el mecanismo que permite que un usuario no-code añada integraciones nuevas, y es una de las piezas de mayor apalancamiento de todo el sistema.
3. **Agente como tool**: `agent.invoke` con el id de otro agente.

Un cuarto origen —tool en Python subida por el usuario— queda explícitamente fuera de V1: exigiría un sandbox real y es la puerta trasera perfecta al Policy Engine.

11. POLICY ENGINE Y SEGURIDAD

11.1 Por qué es estructural y no una capa de validación

El principio "el modelo propone, el runtime dispone" solo es real si es **imposible por construcción** ejecutar una tool sin pasar por la política. Implementación concreta: el ejecutor del bucle no tiene acceso a las funciones de las tools. Solo tiene una referencia al `ToolBroker`, cuyo único método público es:

```
def invoke(self, ctx: RunContext, call: ToolCall) -> ToolResult:
    decision = self.policy.authorize(ctx, call)    # ← no hay ruta alternativa
    ...
```

y `authorize` es lo primero que ocurre, siempre, sin bandera para desactivarlo. Si alguna vez alguien añade un parámetro `skip_policy=True` "para depurar", el sistema ha perdido su propiedad de seguridad. Debe estar prohibido por convención escrita y detectado por un test.

11.2 Modelo de política

Tres niveles que se componen, del más general al más específico:

POLÍTICA DEL SISTEMA	(inmutable, en código: nunca ejecutar <code>`rm -rf /`</code> , nunca salir del workspace, nunca exceder el presupuesto global)
▼ restringe	
POLÍTICA DEL AGENTE	(declarada en la Definición; editable en la UI)
▼ restringe	
POLÍTICA DEL RUN	(puede restringir más aún: un run de prueba en modo dry-run)

Regla: **cada nivel solo puede restringir, nunca ampliar**. Un agente no puede concederse a sí mismo un permiso que el sistema niega.

Forma declarada en la Definición:

```

policies:
  filesystem: { mode: workspace_only, write: true }
  network:    { mode: allowlist, domains: ["*.linkedin.com", "api.serper.dev"] }
  database:   { domain_db: read_write, tables: [companies, signals, leads] }
  shell:      { mode: denied }
  destructive_actions: require_approval
  outbound_messages:      # el caso peligroso de verdad
    mode: require_approval
    max_per_run: 20
    max_per_day: 100
  budget:
    max_steps: 25
    max_tool_calls: 60
    max_tokens: 400000
    max_money_usd: 2.00
    max_wallclock_s: 900
  approval:
    channel: ui          # ui | none
    timeout_s: 3600
    on_timeout: block    # block | deny | proceed(prohibido para destructive)

```

11.3 El presupuesto es una política, no una métrica

Insisto porque es donde se pierde dinero de verdad. El `BudgetManager` se consulta **antes** de cada llamada al LLM y de cada tool call, y decrementa después. Al agotarse, el run termina con `EXHAUSTED` de forma limpia, guardando estado y emitiendo el evento. Un agente L3 sin `max_steps` es, literalmente, un bucle `while True` con tarjeta de crédito.

11.4 Aprobación humana

La cola de aprobaciones es un elemento de producto, no un detalle: es lo que permite operar agentes con acciones irreversibles (enviar mensajes a clientes reales) sin miedo. Diseño mínimo: tabla `approvals` (`run_id`, `tool_call`, `propuesta`, `estado`, `decisor`, `timestamp`), un endpoint, una pantalla y una notificación. El run queda `BLOCKED` con su estado persistido y se reanuda al resolverse. **Que el run sea reanudable desde estado persistido es precisamente el requisito que hace posible esta funcionalidad** — otra razón por la que la máquina de estados no es opcional.

11.5 Secretos

Nunca en la Definición, nunca en el event log, nunca en el contexto del LLM. La Definición referencia `secret_ref: "serper_api_key"`; el runtime lo resuelve desde un almacén local (archivo `.env` cifrado o el keyring del SO) en el momento de ejecutar la tool. El modelo jamás ve el valor. Los argumentos de la tool se redactan antes de persistirse.

12. EVALUATION

12.1 El problema que nadie admite

La mayoría de las tareas de estos agentes **no tienen ground truth**. No existe la respuesta correcta a "encuentra oportunidades de negocio esta semana". Cualquier sistema de evaluación que finja lo contrario produce números bonitos y falsos. La solución es aceptar tres capas de evaluación con costes, latencias y fiabilidades distintas, y no confundirlas jamás.

12.2 Las tres capas

Capa 1 — Métricas mecánicas (gratis, siempre activas, objetivas). Se calculan del event log sin ningún juicio. `completed` / `failed` / `exhausted`, número de pasos, llamadas a herramienta, tasa de error por tool, tokens, coste en dólares, latencia total y por paso, iteraciones hasta terminar, número de

reintentos, número de denegaciones de política. Esto ya responde a la mitad de las preguntas del brief (¿qué herramienta falló? ¿cuánto costó? ¿cuánto tardó?) y no requiere ni un LLM ni un humano.

Capa 2 — Evaluación de calidad por rúbrica (cara, bajo demanda, subjetiva pero comparable). Un conjunto congelado de escenarios (eval sets): entradas fijas + rúbrica + salida esperada cuando exista. Se ejecuta al promocionar una versión, no en cada run. Dos modos: comprobaciones programáticas (¿el output cumple el schema?, ¿el email menciona el nombre de la empresa?, ¿la URL existe?) que deben ser la mayoría; y juez-LLM con rúbrica explícita para lo que no sea programable. Advertencia obligatoria: **el juez-LLM es ruidoso y sesgado hacia respuestas largas**. Úsalo para detectar regresiones groseras, jamás para diferencias de menos de un 10 %.

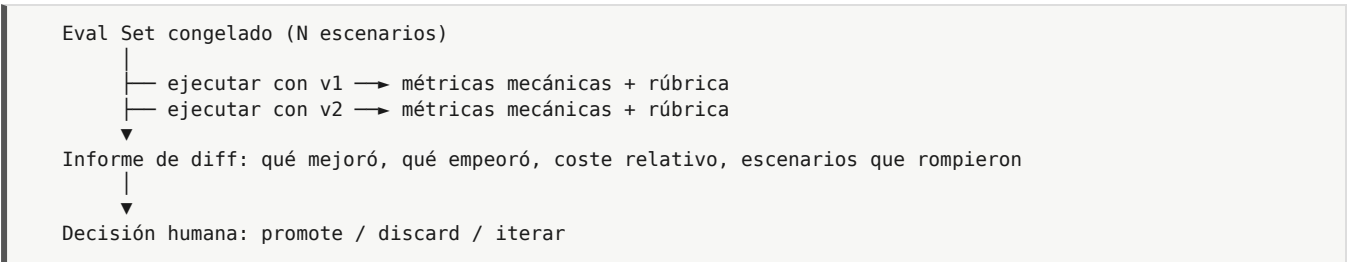
Capa 3 — Outcomes reales (la única que mide el negocio, y llega tarde). `lead_respondió`, `reunión_agendada`, `propuesta_aceptada`, `factura_cobrada`. Se registran de forma asíncrona, con atribución a `run_id` y `agent_version_id`. Requiere que el sistema pueda recibir señales del mundo días después. Es la capa más importante y la que casi todo el mundo omite porque es aburrida.

12.3 Los indicadores mínimos por agente

FIABILIDAD	tasa de terminación limpia · tasa de error por tool · crashes
EFICIENCIA	coste medio por run · coste por resultado útil · pasos por run
CALIDAD	puntuación de rúbrica sobre el eval set congelado
NEGOCIO	resultados producidos por run · coste por outcome · tasa de conversión

La métrica reina, y la que debe presidir el dashboard, es **coste por outcome** (p. ej. dólares por reunión agendada). Es la única que combina las tres capas y la única que puede justificar económicamente todo este sistema.

12.4 Comparación de versiones



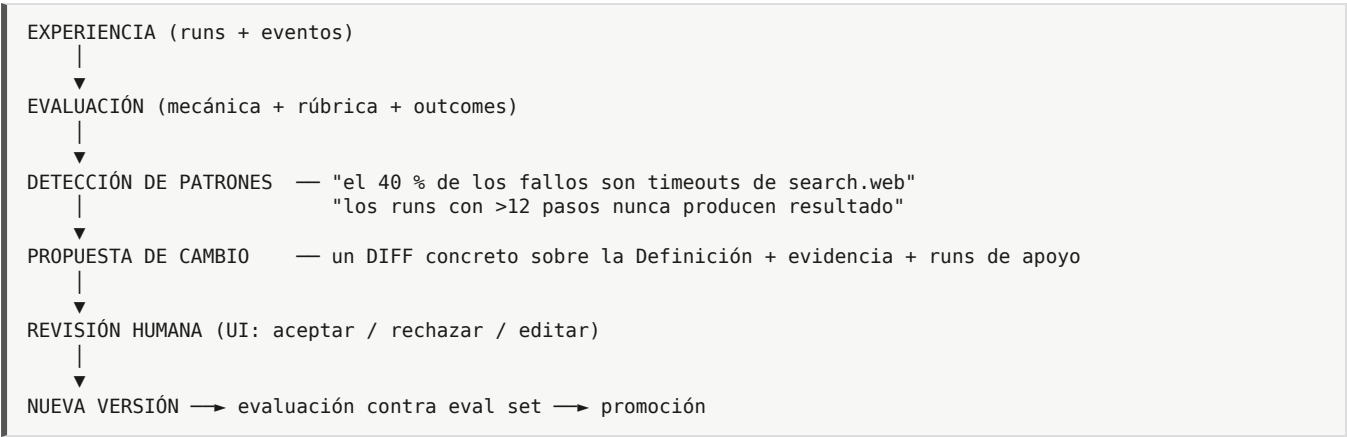
Y en producción, comparación por outcomes con reparto de tráfico (50 % de los runs con v1, 50 % con v2) cuando el volumen lo permita. Esto es barato de implementar (un campo de peso en el despliegue) y muy valioso; merece estar en la arquitectura desde el inicio aunque se active después.

13. AUTO-MEJORA: QUÉ AUTOMATIZAR Y QUÉ NO

13.1 Postura

El brief acierta al no asumir reentrenamiento. Voy más lejos: **en V1 y V2, nada se auto-modifica**. El bucle de mejora produce *propuestas*, no cambios. La razón no es timidez: es que un sistema que edita sus propias políticas y prompts en función de métricas que él mismo genera es un sistema que aprenderá a mejorar la métrica, no el resultado. Con outcomes escasos y ruidosos —que es exactamente tu caso durante los primeros meses—, el sobreajuste es prácticamente seguro.

13.2 El bucle real



13.3 Qué sí puede automatizarse desde el principio

- **La detección:** agregaciones sobre el event log son deterministas y baratas. Automatizar al 100 %.
- **La redacción de la propuesta:** un LLM leyendo métricas y trazas y proponiendo un diff. Automatizar, con salida siempre en forma de diff revisable.
- **La ejecución del eval set** sobre la versión candidata. Automatizar.
- **La adaptación de datos, no de comportamiento:** memoria de largo plazo y entidades del dominio se actualizan continuamente y de forma automática, porque están acotadas por schema y son auditables. Aquí es donde ocurre el "aprendizaje" real y seguro del sistema en los primeros meses.

13.4 Qué requiere humano

Cualquier cambio en: objetivo, políticas, presupuestos, selección de tools, prompts del sistema, criterio de éxito, y la promoción de una versión a activa.

Única excepción admisible más adelante: promoción automática cuando (a) exista un eval set de al menos 30 escenarios, (b) la mejora sea estadísticamente significativa, (c) ninguna métrica de seguridad empeore, y (d) exista rollback automático si los outcomes de producción caen. Antes de tener eso, la promoción automática es una forma elegante de romper agentes que funcionaban.

14. AGENT DEFINITION: EL FORMATO Y LA FUENTE DE VERDAD

14.1 La decisión de formato (y por qué esta y no otra)

El brief pregunta si JSON, YAML, SQLite o combinación. Alternativas evaluadas:

Opción	A favor	En contra	Veredicto
Solo YAML en Git	Diff legible, revisable, versionado gratis	La UI editando ficheros es frágil; sin transacciones; sin consultas	Insuficiente solo
Solo SQLite	Consultas, transacciones, ideal para la UI	Diffs ilegibles; recuperación acoplada al fichero .db; revisión imposible	Insuficiente solo
Solo JSON en disco	Neutro, validable	Mismos problemas que YAML, menos legible para humanos	No

Opción	A favor	En contra	Veredicto
Canónico JSON en SQLite + export YAML determinista a Git	Lo mejor de ambos	Requiere disciplina de sincronización	Elegida

Decisión:

- **Forma canónica:** un documento **JSON** validado por **JSON Schema**, almacenado como texto en la tabla inmutable `agent_versions`, identificado por `sha256` de su serialización canónica (claves ordenadas, sin espacios superfluos). El hash es la identidad real de la versión: dos definiciones idénticas son la misma versión.
- **Forma de trabajo humano y de recuperación:** exportación determinista a `agents/<slug>/v<N>.yaml` en el repositorio Git, más un `agents/<slug>/agent.yaml` que apunta a la versión activa. YAML porque es el que un humano revisa en un diff.
- **Regla de sincronización:** la base de datos manda para *ejecutar*; Git manda para *recuperar y revisar*. Un comando `aap export` y otro `aap import` mantienen la equivalencia, y un test de CI verifica que exportar-importar es idempotente. Si divergen, se reconcilia por hash.

Por qué JSON como canónico y no YAML: porque el schema, la validación, la API y la UI hablan JSON de forma nativa, y porque YAML tiene ambigüedades (el famoso `NO` que se convierte en `false`, versiones sin comillas que se convierten en números) que en un formato canónico son bombas de relojería. YAML se genera *desde* JSON, no al revés.

14.2 El schema (v1)

Ejemplo completo y comentado. Es el artefacto más importante de todo el documento: es el "lenguaje" que el intérprete ejecuta.


```

schema_version: 1
id: demand-hunter          # slug estable
version: 7                  # entero incremental por agente
status: active              # draft | active | archived

identity:
  name: "Demand Hunter"
  description: "Detecta empresas con señales de necesidad de automatización"
  owner: "miguel"
  tags: [ventas, prospección]
  icon: "radar"

goal:
  statement: >
    Encontrar empresas del sector logístico en España que hayan mostrado
    en los últimos 30 días señales de necesidad de automatización de procesos,
    y registrarlas como oportunidades cualificadas.
  success_criteria:          # verificable, no prosa
    - type: metric
      expr: "signals_qualified >= 5"
    - type: metric
      expr: "duplicates_created == 0"
  failure_criteria:
    - type: metric
      expr: "tool_error_rate > 0.5"

runtime:
  autonomy_level: 3          # L0..L4
  max_iterations: 15
  concurrency: 1
  resumable: true

brain:
  # NUNCA un modelo físico concreto
  primary: { capability: standard, temperature: 0.2 }
  reasoning: { capability: heavy, temperature: 0.4, use_for: [plan, replan] }
  cheap: { capability: cheap, use_for: [classify, extract] }
  system_prompt_ref: prompts/demand_hunter/system.md # versionado con la definición
  response_format: tool_calling # tool_calling | json_schema | text

tools:
  - id: search.web          ; config: { max_results: 10 }
  - id: http.get
  - id: llm.extract
  - id: db.upsert           ; config: { tables: [companies, signals] }
  - id: memory.search
  - id: memory.write

knowledge:
  sources:
    - { id: icp, type: document, path: knowledge/icp_logistica.md }
    - { id: senales, type: document, path: knowledge/taxonomia_senales.md }
  retrieval: { mode: top_k, k: 4, min_score: 0.5 }

memory:
  long_term:
    enabled: true
    max_entries: 500
    types: [empresa_descartada, patron_senal, contacto_verificado]
    retrieval: { k: 6, recency_weight: 0.3 }
  state_schema:
    # el estado permitido, tipado
    fase: { type: string, enum: [buscar, filtrar, enriquecer, registrar] }
    empresas_vistas: { type: integer, default: 0 }
    senales_validas: { type: integer, default: 0 }
    ultimo_cursor: { type: string, default: "" }

policies:
  # ver §11.2 – se omite aquí por brevedad
  network: { mode: allowlist, domains: ["*.serper.dev", "*.linkedin.com"] }
  database: { domain_db: read_write, tables: [companies, signals] }
  shell: { mode: denied }
  outbound_messages: { mode: denied }
  budget: { max_steps: 25, max_tokens: 400000, max_money_usd: 2.00, max_wallclock_s: 900 }

workflow:
  # opcional; si falta, bucle puro del nivel L
  type: loop
  graph: null

triggers:

```

```

- { type: schedule, cron: "0 7 * * 1-5", timezone: "America/Caracas" }
- { type: manual }
- { type: api }

io:
  input_schema: { sector: {type: string}, region: {type: string} }
  output_schema: { qualified: {type: array}, report: {type: string} }

evaluation:
  eval_set_ref: evals/demand_hunter_v1.jsonl
  metrics: [completion_rate, cost_per_run, signals_per_run, duplicate_rate]
  outcome_links: [lead_replied, meeting_booked] # atribución a largo plazo

limits:
  max_runs_per_day: 24
  max_concurrent_runs: 1

```

14.3 Propiedades que este schema garantiza

1. **No contiene código.** Ni una expresión ejecutable arbitraria. Las `expr` de los criterios son un mini-lenguaje de comparación sobre variables de estado, evaluado por un evaluador seguro y limitado (no `eval()` de Python: eso sería una vía de ejecución arbitraria que rompe todo el modelo de seguridad).
2. **No contiene secretos ni endpoints.** `capability: heavy` es una necesidad; a qué máquina se traduce lo decide el entorno.
3. **Es enteramente representable en formularios.** Cada bloque del schema es una pantalla de la UI. Esa correspondencia 1:1 es deliberada, y es lo que hace que el no-code sea real y no una fachada.
4. **Es diffeable.** Dos versiones producen un diff YAML legible que un humano puede revisar en treinta segundos.

15. VISUAL BUILDER: EL NO-CODE COMO PROPIEDAD ARQUITECTÓNICA

15.1 La corrección de criterio más importante de esta sección

El brief exige que el no-code no sea un añadido posterior. **Estoy de acuerdo con el principio y en desacuerdo con la implicación de calendario.**

El requisito arquitectónico real es: *"la Definición debe ser completa, declarativa y suficiente para configurar cualquier comportamiento normal, sin escribir código"*. Ese requisito se cumple en el hito M1, escribiendo el schema, mucho antes de que exista un solo píxel de interfaz. Si el schema es completo, la UI es un problema resuelto: son formularios generados sobre un schema.

Lo que **no** conviene es construir el canvas visual de workflows en la V1. Razones concretas:

- El vocabulario de nodos aún no está estabilizado; dibujarlo antes de saber qué nodos existen produce un canvas que hay que rehacer.
- Un canvas es una de las piezas de UI más caras que existen (layout, conexiones, validación de grafos, undo, zoom, serialización). Semanas de trabajo.
- Los primeros cinco agentes no necesitan grafo: necesitan un bucle con buenas tools. El grafo aporta valor a partir del momento en que compones varios agentes.

Plan en tres etapas:

Etapas	Qué se construye	Cuándo
B0	Ninguna UI. La Definición se edita en YAML y se valida por CLI. La API completa ya existe.	M1-M6

Etapa	Qué se construye	Cuándo
B1	UI de formularios generada desde el JSON Schema: 12 pantallas, sin canvas. Cubre el 90 % del valor no-code.	M10
B2	Canvas visual de workflow para composición de agentes y ramificación explícita.	Post-V1

Esto no viola el principio del brief: la definición declarativa —que es donde el no-code se gana o se pierde— está desde el día uno. Lo que se difiere es el lienzo, no la capacidad.

15.2 Las pantallas y sus controles

Correspondencia directa con los bloques del schema. Marco qué es visible para el usuario normal (**N**) y qué solo en modo avanzado (**A**).

Pantalla	Controles	Editable	Automático
Overview (N)	Nombre, estado, versión activa, últimos runs, coste 7 días, botón Ejecutar/Pausar/Duplicar	Estado	Métricas
Identity (N)	Nombre, descripción, icono, etiquetas, dueño	Todo	slug
Goal (N)	Objetivo en lenguaje natural; constructor visual de criterios de éxito (variable · operador · valor)	Todo	Traducción a expr
Brain (N/A)	Selector de "calidad de razonamiento" (Rápido / Equilibrado / Profundo) → mapea a capabilities. Temperatura y prompt de sistema solo en A	Preset	Modelo físico, routing
Tools (N)	Catálogo con interruptores, descripción legible, indicador de riesgo (verde/ámbar/rojo por <code>side_effects</code>), configuración por tool	Selección	Schemas, permisos derivados
Memory (N/A)	Activar memoria de largo plazo, tipos permitidos, límite. <code>state_schema</code> solo en A	Límites	Recuperación
Knowledge (N)	Subida de ficheros, lista, previsualización, reindexado	Documentos	Chunking, embeddings
Policies (N)	Interruptores en lenguaje humano: "puede navegar por internet (dominios: ...)", "puede enviar mensajes (requiere mi aprobación)", "presupuesto máximo por ejecución: 2 \$". Reglas crudas en A	Todo	Política del sistema
Workflow (A, B2)	Canvas o, en B1, selector de nivel de autonomía con explicación en texto y coste estimado	Nivel	Estructura del bucle
Schedules (N)	Constructor de horarios sin cron ("cada día laborable a las 7:00"), límites diarios	Todo	Traducción a cron
Inputs/Outputs (A)	Editor de schemas de entrada/salida	Todo	—
Evaluation (N/A)	Métricas a seguir, eval set (subir/generar), enlaces a outcomes	Selección	Cálculo
Runs / Logs (N)	Lista de ejecuciones, inspector de traza paso a paso, coste, errores	—	Todo
Versions (N)	Historial, diff visual entre versiones, promocionar, revertir, archivar	Acciones	Diff
Deploy (N)	Activar/desactivar, reparto de tráfico entre versiones, entorno	Todo	—

15.3 Cómo se evita que el modo avanzado contamine el no-code

- Un único interruptor global "Modo avanzado" en el perfil, no por pantalla.
- Las pantallas avanzadas y los campos avanzados están **ocultos**, no deshabilitados; un campo gris que no puedes tocar genera más ansiedad que uno que no existe.
- Todo campo avanzado tiene **un valor por defecto sensato que funciona**. El usuario no-code nunca debe encontrarse con un agente que no arranca porque no rellenó algo que no ve.
- Existe siempre un botón "Ver definición (YAML)" en modo lectura, incluso para usuarios normales: es la vía de escape que hace que el sistema sea comprensible y depurable, y refuerza el modelo mental de que la UI edita un documento.

15.4 Cómo funcionaría técnicamente el canvas (etapa B2)

Para dejarlo especificado aunque se difiera:

- El canvas serializa a `workflow.graph` dentro de la misma Definición: `{nodes: [...], edges: [...]}`. **No genera código**. El runtime interpreta el grafo.
- Tipos de nodo, deliberadamente pocos: `trigger`, `agent` (ejecuta un agente con un bucle interno), `tool` (una llamada determinista), `branch` (condición sobre estado), `parallel`, `human_approval`, `end`.
- El grafo es un **DAG con ciclos permitidos solo hacia atrás y con contador de iteraciones obligatorio**. Sin esa restricción, el canvas se convierte en un lenguaje de programación mal diseñado y con bucles infinitos.
- Validación en el cliente y de nuevo en el servidor: nodos alcanzables, tipos compatibles, presupuesto total acotado.
- **La distinción clave**: el grafo es la orquestación *determinista* exterior; la autonomía vive *dentro* de los nodos `agent`. Mezclar ambos —dejar que el LLM salte por el grafo libremente— produce un sistema imposible de depurar. Este es el error de diseño más común en las plataformas de agentes visuales, y merece la pena evitarlo por escrito.

16. AGENT FACTORY, DUPLICACIÓN Y VERSIONADO

16.1 Qué es runtime y qué es factory

FACTORY (control plane, sin código de ejecución)	RUNTIME (ejecución)
plantillas de agente	cargar versión
wizard de creación	construir contexto
clonar agente	ejecutar bucle
editar borrador · validar · guardar versión	autorizar y llamar tools
diff entre versiones	mantener estado
promocionar / revertir / archivar	emitir eventos
lanzar evaluación	aplicar presupuesto
programar	terminar y persistir

La Factory **no toca el runtime**. Si algún día una funcionalidad de la Factory exige cambiar el runtime, es señal de que el schema es insuficiente: se arregla el schema, no el runtime.

16.2 Plantillas

Una plantilla es una Definición incompleta más metadatos de wizard: qué campos pregunta, en qué orden y con qué ayuda. Se guardan en `templates/` en el repo, versionadas con el código porque evolucionan con las capacidades del runtime.

Plantillas iniciales previstas: `blank`, `researcher` (L2), `classifier` (L1), `ingestor` (L0), `outreach` (L1 con aprobación obligatoria).

16.3 Duplicación

```
POST /agents/{id}/duplicate {new_name}
→ copia la definición de la versión activa
→ nuevo slug, versión 1, status=draft
→ NO copia: runs, eventos, memorias, estado
→ SÍ copia (opcional, con confirmación explícita): knowledge, eval set
```

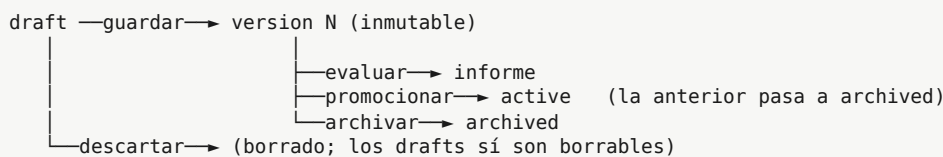
Que la memoria de largo plazo **no** se copie por defecto es una decisión consciente: heredar las creencias de otro agente sin heredar su contexto es la vía más rápida a comportamientos inexplicables. Debe ser una casilla marcada a propósito.

16.4 Versionado: la decisión

El brief pregunta si Git, base de datos, snapshots o combinación. **Combinación, con roles claramente separados:**

- **Base de datos (agent_versions)** = el registro autoritativo para ejecutar. Inmutable, con hash de contenido, con estado (draft|active|archived) y con timestamps. Es lo que consulta el runtime, y es transaccional.
- **Git** = el registro autoritativo para revisar y recuperar. Cada versión promocionada se exporta a YAML y se commitea automáticamente con un mensaje generado (agent(demand-hunter): v7 – amplía allowlist, sube max_steps a 25). Aporta diff, historia, revisión, respaldo remoto y recuperación total si el .db se pierde.
- **Snapshots de estado** = distintos de las versiones. Copia de seguridad periódica de las tres bases de datos, que se guarda como artefacto, no en Git.

Ciclo de vida de una versión:



Regla: **siempre existe exactamente una versión activa por agente** (o dos con reparto de tráfico durante un experimento). El rollback es promocionar una versión anterior: una operación de un clic y de riesgo cero, porque las versiones son inmutables.

17. LLM INTERFACE Y MODEL ROUTING

17.1 El contrato

```
class LLMInterface(Protocol):
    def complete(self, req: CompletionRequest) -> CompletionResult: ...

# CompletionRequest
# messages: list[Message]
# tools: list[ToolSpec] | None # el runtime decide qué ve el modelo
# response_schema: dict | None # salida estructurada obligatoria
# capability: "cheap" | "standard" | "heavy" | "coding" | "embedding"
# max_tokens, temperature, timeout_s, stop
# budget_ctx: referencia al BudgetManager del run

# CompletionResult
# text: str | None
# tool_calls: list[ToolCall]
# usage: {prompt_tokens, completion_tokens, cost_usd, latency_ms}
# model_used: str # el físico, para la traza
# finish_reason: str
```

El agente jamás nombra un modelo. Nombra una **capacidad**. El mapeo capacidad → modelo físico vive en la configuración del entorno:

```
# config/models.yaml – ESTO NO ES PARTE DE NINGUNA DEFINICIÓN DE AGENTE
providers:
  qwen_local:
    kind: openai_compatible
    base_url: "${VLLM_URL}" # instancia GPU efímera; puede no existir
    model: "Qwen3-Coder-Next"
    cost_per_1k: { in: 0, out: 0 }
  api_fast:
    kind: openai_compatible
    base_url: "..."
    model: "..."
    cost_per_1k: { in: 0.0002, out: 0.0008 }
  mock:
    kind: mock # para tests: determinista y gratis

routing:
  cheap: [api_fast, qwen_local, mock] # lista ordenada = cadena de degradación
  standard: [qwen_local, api_fast, mock]
  heavy: [qwen_local, api_fast]
  coding: [qwen_local]
  embedding: [local_embed]

policies:
  on_unavailable: next_in_chain # next_in_chain | queue | fail
  max_retries: 2
```

17.2 Por qué la cadena de degradación es una pieza de arquitectura y no un detalle

Porque tu infraestructura de inferencia es intermitente por diseño. **Cuando la GPU no está encendida, el sistema no puede caerse.** Las opciones al fallar deben ser declarativas: pasar al siguiente proveedor, encolar el run hasta que haya capacidad, o fallar limpiamente. Un runtime que asume que el modelo siempre responde es un runtime que no sobrevive a tu propia infraestructura.

17.3 Routing: contrato ahora, inteligencia después

En V1 el router es: *toma la capacidad declarada, recorre la cadena, devuelve el primer proveedor sano.* Nada más. Los ganchos previstos para después —y ya presentes en la firma— son señales de tarea (`task_type`, longitud estimada del prompt, criticidad) que permitirían reglas del tipo "si el prompt supera 30 000 tokens, salta a un modelo con contexto largo" o "clasificación siempre a cheap". Se añaden cuando haya datos de coste que lo justifiquen; construirlo antes es optimizar a ciegas.

17.4 La regla que ahorra más dinero de todas

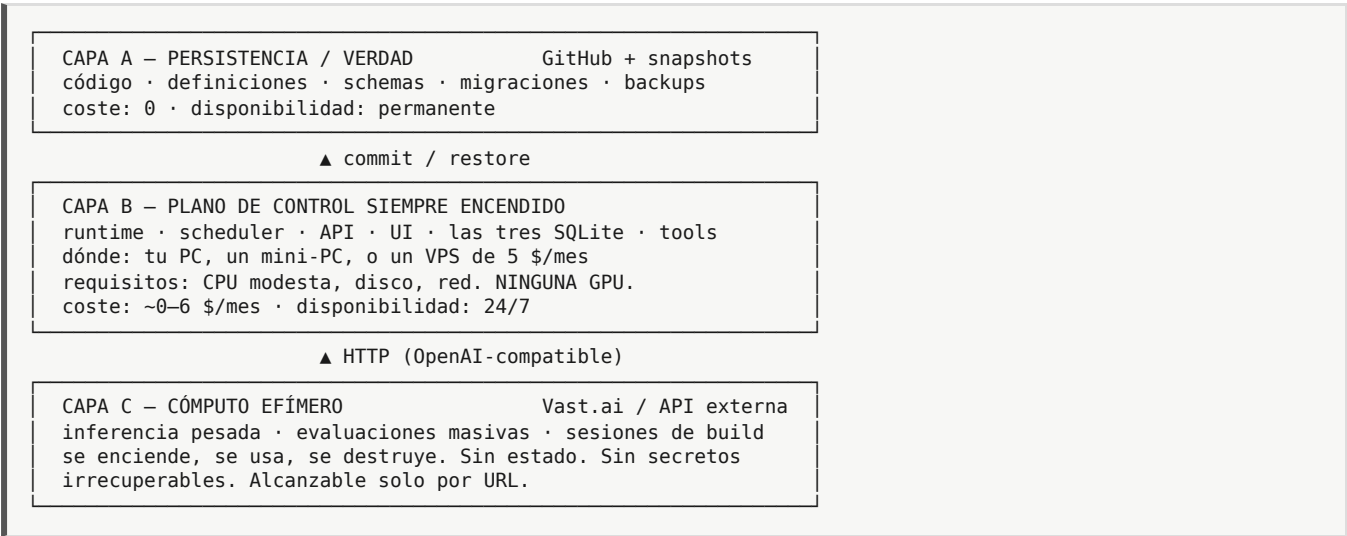
Elegir el nivel de autonomía y la capacidad de modelo más bajos que resuelvan la tarea. La mayor parte del gasto en estos sistemas viene de usar un modelo grande en un bucle iterativo para tareas que un modelo pequeño resolvería en una sola llamada. Esta regla debe estar visible en la UI, en la pantalla Brain, con estimación de coste por ejecución en tiempo real.

18. CLOUD VS LOCAL: LA FRONTERA OPERACIONAL

18.1 La corrección: son tres capas, no dos

El brief plantea `CLOUD = cómputo` y `LOCAL/GITHUB = persistencia`. Es correcto pero incompleto, y la pieza que falta es la que decide si el proyecto funciona en la práctica: **¿dónde vive el proceso que debe estar encendido a las 7:00 de la mañana para ejecutar el Demand Hunter?**

No puede ser la GPU (es efímera y cara). No puede ser solo GitHub (no ejecuta nada). Falta un tercer elemento:



La consecuencia práctica más importante: el runtime nunca se despliega dentro de la instancia GPU. La instancia GPU solo sirve un endpoint de inferencia. Si algún día el runtime se ejecuta en la GPU (por conveniencia durante una sesión de desarrollo), debe ser una ejecución desechable, con las bases de datos montadas desde fuera o sincronizadas al terminar.

18.2 Qué va dónde, exhaustivo

Elemento	Capa	Motivo
Código del runtime	A (verdad) → B (ejecución)	Git es la verdad; se ejecuta en el plano de control
Definiciones de agentes	A + B	Canónico en <code>control.db</code> (B), exportado a Git (A)
<code>control.db</code> , <code>runtime.db</code> , <code>domain.db</code>	B, con snapshot a A	Deben estar donde corre el scheduler
Knowledge (documentos)	A + B	En Git si son pequeños; en disco de B si son grandes, con backup
Índices de embeddings	B	Regenerables; no se respaldan, se reconstruyen
Secretos	B únicamente	Nunca en Git, nunca en la GPU
Servidor vLLM + pesos del modelo	C	Es lo único que necesita GPU

Elemento	Capa	Motivo
Evaluaciones masivas / benchmarks	C	Cómputo intensivo y puntual
Sesiones de desarrollo asistidas por modelo grande	C	El uso legítimo de la GPU durante la construcción
Artefactos de run (ficheros generados)	B, con retención	Se limpian a los 30 días

18.3 Cuándo encender la GPU (criterio económico)

Encender GPU tiene sentido cuando el coste por hora dividido entre el trabajo realizado es inferior a la alternativa por API. Con volúmenes bajos —tu caso durante los primeros meses— **una API externa por tokens es casi siempre más barata y siempre más fiable que una GPU alquilada por horas**, porque pagas cero cuando no la usas y no pagas el arranque, la descarga de pesos ni el tiempo muerto.

Recomendación concreta y contraria al plan inicial: **difiere vLLM y Qwen local hasta el hito M12**. Desarrolla y opera con (a) el provider `mock` para tests y (b) una API compatible con OpenAI para lo real. Introduce el modelo local cuando se cumpla al menos una de estas tres condiciones: el gasto mensual en API supere el coste de la GPU; necesites procesar datos que no pueden salir de tu infraestructura; o necesites un modelo específico que no está disponible por API. La abstracción del LLM Interface hace que ese cambio sea, literalmente, editar `config/models.yaml`. Eso es el valor de la abstracción, y por eso hay que construirla primero y el modelo local después.

19. DOCKER: ARQUITECTURA DE EJECUCIÓN

19.1 Principio

Docker es un mecanismo de reproducibilidad y de aislamiento, no una parte de la lógica del agente. Ningún componente del sistema debe "saber" que está en un contenedor.

19.2 Qué va dentro y qué va fuera

```

HOST (capa B)
├── /srv/aap/data/          ← VOLUMEN. Nunca dentro de la imagen.
│   ├── control.db runtime.db domain.db knowledge/ artifacts/ .secrets
│   └── docker-compose.yml
├──
├── contenedor: aap-api      (FastAPI + UI estática)      puerto 8080
├── contenedor: aap-worker   (ejecutor de runs + scheduler)
├── contenedor: aap-tools-sbx (opcional, fase 2: sandbox para tools peligrosas)
├──
└──
HOST GPU (capa C, remoto y efímero)
├── contenedor: vllm-server  puerto 8000 → expuesto por túnel/IP a la capa B
└──

```

Decisiones:

- **API y worker separados** desde el principio. Un bucle de agente que bloquea el servidor web es una fuente inagotable de problemas, y separarlos después obliga a reescribir la gestión de estado. Comparten la imagen y el código; difieren en el comando de arranque.
- **El modelo nunca en el mismo compose** que el runtime. Distinta máquina, distinto ciclo de vida, distinta cuenta de coste.
- **Estado siempre en volumen montado**. Regla verificable: `docker compose down --rmi all` seguido de `up` debe dejar el sistema exactamente como estaba.
- **SQLite en volumen Docker**: usar bind mount a un directorio del host, no un volumen nombrado, y **nunca** sobre un sistema de ficheros de red (NFS/SMB corrompe SQLite). Modo WAL activado.

- El worker corre como usuario no-root y solo tiene montado su directorio de workspace en escritura.

19.3 La propiedad de reconstrucción (requisito explícito)

```
git clone git@github.com:<user>/aap.git && cd aap
cp .env.example .env && $EDITOR .env          # secretos (único paso manual)
scripts/restore.sh backups/latest.tar.age      # restaura las tres DB y knowledge
docker compose up -d
scripts/healthcheck.sh                        # verifica: API, worker, DB, modelo
```

Esto debe funcionar en una máquina virgen en menos de diez minutos, y debe probarse de verdad —no razonarse— en el hito M8. Un procedimiento de restauración que nunca se ha ejecutado no existe.

20. ESTRUCTURA DEL REPOSITORIO

20.1 Propuesta, con justificación de lo que se elimina

El brief propone 16 directorios de primer nivel. Rechazo esa granularidad: `state`, `events`, `memory` y `policies` no son subsistemas independientes, son módulos del núcleo, y elevarlos a directorios raíz sugiere una separación que no existe y fomenta dependencias circulares.

```
aap/
├── README.md
├── docs/
│   ├── ARCHITECTURE.md          ← este documento
│   ├── DECISIONS/              ← ADRs numerados; un fichero por decisión
│   └── RUNBOOK.md              ← operación: backup, restore, incidencias
├── pyproject.toml
├── docker-compose.yml · Dockerfile · .env.example
├── src/aap/
│   ├── core/                   ← EL INTÉRPRETE. Sin conocimiento de dominio.
│   │   ├── definition/         schema.json, validate.py, migrate.py, export.py
│   │   ├── runtime/            executor_l0..l4, context.py, state.py, budget.py
│   │   ├── llm/                interface.py, router.py, providers/
│   │   ├── tools/              broker.py, registry.py, spec.py
│   │   ├── policy/             engine.py, rules.py, approvals.py
│   │   ├── memory/             longterm.py, knowledge.py, retrieval.py
│   │   ├── events/             log.py, types.py
│   │   └── evaluation/         metrics.py, rubric.py, compare.py
│   ├── tools/                  ← IMPLEMENTACIONES concretas. Crece sin límite.
│   │   ├── builtin/            http.py, search.py, db.py, fs.py, llm_extract.py
│   │   └── declarative/        *.yaml (tools HTTP sin código)
│   ├── domain/                 ← EL NEGOCIO. Entity Store y su lógica.
│   │   ├── models.py           companies, signals, leads, contacts, outreach
│   │   ├── dedup.py            deduplicación semántica y por clave natural
│   │   └── outcomes.py         registro y atribución de resultados
│   ├── api/                    ← FastAPI: routers, schemas de request/response
│   │   ├── factory/            ← plantillas, clonado, diff, promoción
│   │   ├── scheduler/          ← triggers, cron, cola de runs
│   │   └── cli/                 ← aap run|validate|export|import|eval|backup
├── agents/                     ← DEFINICIONES exportadas (una carpeta por agente)
│   └── demand-hunter/ agent.yaml · v1.yaml ... v7.yaml · prompts/ · knowledge/
├── templates/                  ← plantillas de la factory
├── evals/                       ← eval sets congelados (.jsonl) y rúbricas
├── migrations/                 ← migraciones de SQLite y del schema de definición
├── ui/                          ← frontend (etapa B1)
├── scripts/                     ← bootstrap, backup, restore, healthcheck
└── tests/                       ← unit, integration, e2e (con provider mock)
```

20.2 Las tres reglas de dependencia (verificables en CI)

1. `core/` **no importa** de `tools/`, `domain/`, `api/` ni `factory/`. Solo de sí mismo y de la librería estándar. Esta es la regla que mantiene el intérprete genérico; su violación es el primer síntoma de que el sistema se está convirtiendo en un monolito.
2. `domain/` no importa de `core/runtime`. El almacén de dominio existe con independencia de los agentes.
3. `api/` y `cli/` son clientes: pueden importar de todo, y nadie importa de ellos.

Un test que recorre los imports y falla si se rompe alguna de las tres. Cuesta veinte líneas y salva el proyecto.

21. MODELO DE DATOS (SQLite)

21.1 Tres bases de datos, no una

Fichero	Contiene	Volumen	Backup	Se puede perder
<code>control.db</code>	agentes, versiones, plantillas, memorias, aprobaciones, secretos-ref, config	Pequeño	Cada cambio + diario	No (aunque Git lo reconstruye casi todo)
<code>runtime.db</code>	runs, eventos, tool_calls, estado, evaluaciones	Muy grande, alta escritura	Diario, con retención 90 días	Tolerable (se pierde historia, no capacidad)
<code>domain.db</code>	entidades de negocio y outcomes	Grande y creciente	Diario, sin borrado	Nunca — es el activo real

Motivo de la separación: ciclos de vida, criticidad y patrones de escritura radicalmente distintos. El event log escribe miles de filas por run y no debe competir por el bloqueo de escritura con el registro de agentes; el dominio debe poder respaldarse y migrarse a Postgres sin arrastrar la historia de trazas. Todas en modo **WAL**, un único proceso escritor por fichero.

21.2 Esquema mínimo

```
-- ===== control.db =====
CREATE TABLE agents (
  id TEXT PRIMARY KEY,          -- slug
  name TEXT NOT NULL,
  owner TEXT,
  active_version_id TEXT,       -- FK agent_versions
  status TEXT NOT NULL,        -- active | paused | archived
  created_at TEXT, updated_at TEXT
);

CREATE TABLE agent_versions (    -- IMMUTABLE
  id TEXT PRIMARY KEY,          -- uuid
  agent_id TEXT NOT NULL REFERENCES agents(id),
  version INTEGER NOT NULL,
  definition_json TEXT NOT NULL, -- documento canónico
  content_hash TEXT NOT NULL,    -- sha256 de la serialización canónica
  schema_version INTEGER NOT NULL,
  status TEXT NOT NULL,         -- draft | active | archived
  created_by TEXT, created_at TEXT,
  notes TEXT,                  -- por qué se creó esta versión
  UNIQUE(agent_id, version)
);

CREATE TABLE memories (        -- memoria de largo plazo, CURADA
  id TEXT PRIMARY KEY,
  agent_id TEXT NOT NULL,
  type TEXT NOT NULL,          -- debe estar en definition.memory.long_term.types
  subject TEXT,                -- entidad a la que se refiere
  content TEXT NOT NULL,
  confidence REAL,
  source_run_id TEXT,          -- PROCEDENCIA OBLIGATORIA
  embedding BLOB,
  created_at TEXT, last_used_at TEXT, use_count INTEGER DEFAULT 0,
  expires_at TEXT
);

CREATE TABLE approvals (
  id TEXT PRIMARY KEY, run_id TEXT NOT NULL, agent_id TEXT,
  tool_id TEXT, payload_json TEXT, risk TEXT,
  status TEXT NOT NULL,        -- pending | approved | rejected | expired
  decided_by TEXT, decided_at TEXT, created_at TEXT
);

CREATE TABLE schedules (
  id TEXT PRIMARY KEY, agent_id TEXT NOT NULL, cron TEXT, timezone TEXT,
  enabled INTEGER, last_fired_at TEXT, next_fire_at TEXT
);

-- ===== runtime.db =====
CREATE TABLE runs (
  id TEXT PRIMARY KEY,
  agent_id TEXT NOT NULL,
  agent_version_id TEXT NOT NULL, -- ATRIBUCIÓN: nunca solo agent_id
  trigger TEXT,                  -- schedule | manual | api | agent
  parent_run_id TEXT,            -- para agent.invoke
  status TEXT NOT NULL,          -- queued|running|blocked|completed|
                                -- failed|exhausted|cancelled|crashed

  input_json TEXT, output_json TEXT,
  started_at TEXT, finished_at TEXT,
  steps INTEGER DEFAULT 0, tool_calls INTEGER DEFAULT 0,
  tokens_in INTEGER DEFAULT 0, tokens_out INTEGER DEFAULT 0,
  cost_usd REAL DEFAULT 0, latency_ms INTEGER,
  termination_reason TEXT, error TEXT
);
CREATE INDEX idx_runs_agent_time ON runs(agent_id, started_at DESC);

CREATE TABLE events (          -- APPEND-ONLY. Traza + log + auditoría.
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  run_id TEXT NOT NULL, seq INTEGER NOT NULL,
  ts TEXT NOT NULL,
  type TEXT NOT NULL,           -- run.started, llm.called, tool.result, ...
  level TEXT NOT NULL,          -- DEBUG|INFO|WARN|ERROR|AUDIT
  step INTEGER,
  payload_json TEXT,            -- ya redactado
  UNIQUE(run_id, seq)
```

```

);
CREATE INDEX idx_events_run ON events(run_id, seq);

CREATE TABLE tool_calls (
    id TEXT PRIMARY KEY, run_id TEXT NOT NULL, step INTEGER,
    tool_id TEXT NOT NULL, args_json TEXT, result_json TEXT,
    status TEXT, -- ok | error | denied | timeout | pending_approval
    policy_decision TEXT, error TEXT,
    latency_ms INTEGER, started_at TEXT
);

CREATE TABLE run_state (
    run_id TEXT PRIMARY KEY, state_json TEXT NOT NULL,
    version INTEGER NOT NULL, updated_at TEXT -- version = control optimista
);

CREATE TABLE evaluations (
    id TEXT PRIMARY KEY,
    agent_version_id TEXT NOT NULL,
    kind TEXT NOT NULL, -- mechanical | rubric | outcome
    eval_set TEXT, run_id TEXT,
    metrics_json TEXT NOT NULL, score REAL,
    created_at TEXT
);

-- ===== domain.db ===== (ejemplo para el vertical de adquisición)
CREATE TABLE companies (
    id TEXT PRIMARY KEY, name TEXT NOT NULL, domain TEXT UNIQUE,
    sector TEXT, country TEXT, size_band TEXT,
    natural_key TEXT UNIQUE, -- clave de deduplicación determinista
    embedding BLOB, -- deduplicación semántica
    first_seen_at TEXT, last_seen_at TEXT, source_run_id TEXT
);

CREATE TABLE signals (
    id TEXT PRIMARY KEY, company_id TEXT REFERENCES companies(id),
    type TEXT, evidence_url TEXT, excerpt TEXT,
    score REAL, observed_at TEXT,
    source_run_id TEXT, agent_version_id TEXT -- ATRIBUCIÓN
);

CREATE TABLE opportunities (
    id TEXT PRIMARY KEY, company_id TEXT, status TEXT,
    expected_value REAL, reason TEXT,
    created_at TEXT, source_run_id TEXT, agent_version_id TEXT
);

CREATE TABLE outcomes (
    id TEXT PRIMARY KEY, -- LA TABLA MÁS IMPORTANTE DEL SISTEMA
    kind TEXT NOT NULL, -- lead_replied | meeting_booked | deal_won | ...
    entity_type TEXT, entity_id TEXT,
    value_usd REAL,
    occurred_at TEXT, -- puede ser MUY posterior al run
    run_id TEXT, agent_version_id TEXT, -- ATRIBUCIÓN
    recorded_by TEXT, recorded_at TEXT
);

```

21.3 Nota sobre el crecimiento y la migración

`events` crecerá deprisa (miles de filas por run). Política de retención desde el día uno: eventos `DEBUG` se purgan a los 7 días, `INFO` a los 90, `AUDIT` y `ERROR` nunca. Las métricas agregadas por run viven en `runs`, de modo que purgar eventos no destruye la capacidad de evaluar.

Disparador de migración a PostgreSQL (escríbelo ahora, obedéclo después): más de un proceso escritor sobre la misma base, o runs concurrentes sostenidos por encima de ~20, o necesidad de acceso desde varias máquinas. Hasta entonces, SQLite es más rápido, más simple y más fiable que cualquier alternativa. El código debe acceder a la base a través de una capa de repositorios finos para que la migración sea un trabajo de días y no de meses.

22. API

22.1 Qué es FastAPI aquí

El brief pregunta si FastAPI es control plane, API de agente, backend de UI o runtime API. Respuesta: **control plane + backend de UI, que son la misma cosa** (principio P6). **No** es la API del runtime: el runtime es una librería en proceso que usa el worker. Exponer el runtime por HTTP en V1 sería distribuir un sistema que cabe en una máquina.

22.2 Endpoints

— AGENTES —		
GET	/agents	lista + estado + métricas resumen
POST	/agents	crear (desde plantilla o en blanco)
GET	/agents/{id}	detalle + versión activa
PATCH	/agents/{id}	estado (pausar/reanudar), metadatos
DELETE	/agents/{id}	archivar (nunca borrado físico)
POST	/agents/{id}/duplicate	→ nuevo agente en draft
— VERSIONES —		
GET	/agents/{id}/versions	
POST	/agents/{id}/versions	crear versión desde un draft (valida schema)
GET	/agents/{id}/versions/{v}	
GET	/agents/{id}/versions/{a}/diff/{b}	
POST	/agents/{id}/versions/{v}/promote	→ active (archiva la anterior)
POST	/agents/{id}/versions/{v}/archive	
POST	/definitions/validate	valida sin guardar (lo usa la UI en vivo)
— EJECUCIÓN —		
POST	/agents/{id}/runs	lanzar {input, version?, dry_run?} → 202 run_id
GET	/runs	filtros: agent, status, rango, versión
GET	/runs/{id}	estado, coste, resultado
GET	/runs/{id}/events	traza paginada (el inspector)
GET	/runs/{id}/state	
POST	/runs/{id}/cancel	
POST	/runs/{id}/resume	reanuda un run BLOCKED
— APROBACIONES —		
GET	/approvals?status=pending	
POST	/approvals/{id}/decide	{approve reject, comment}
— MEMORIA Y CONOCIMIENTO —		
GET	/agents/{id}/memories	listar / buscar / borrar (curación humana)
DELETE	/agents/{id}/memories/{mid}	
POST	/agents/{id}/knowledge	subir documento → indexa
DELETE	/agents/{id}/knowledge/{kid}	
— EVALUACIÓN Y RESULTADOS —		
POST	/agents/{id}/evaluate	{version, eval_set} → job
GET	/agents/{id}/metrics	?from&to&version → serie de métricas
GET	/agents/{id}/compare?a=&b=	informe de comparación de versiones
POST	/outcomes	registrar outcome externo (webhook/manual)
— CATÁLOGO Y SISTEMA —		
GET	/tools	catálogo con schemas y nivel de riesgo
GET	/templates	
GET	/schema/agent-definition	JSON Schema (la UI genera formularios de aquí)
GET	/health	api, worker, db, providers de modelo

22.3 Tres decisiones de contrato

1. **Los runs son asíncronos siempre.** POST /runs devuelve 202 con un run_id. Progreso por polling en V1 (GET /runs/{id}), con SSE en la UI cuando moleste. Un endpoint síncrono que espera a que termine un agente L3 es una promesa de timeouts.
2. **La UI genera sus formularios desde GET /schema/agent-definition.** Consecuencia: extender el schema añade controles a la UI sin tocar el frontend. Es lo que hace que el no-code escale con el sistema en lugar de quedarse atrás.

3. **dry_run en todos los endpoints de ejecución.** Ejecuta el bucle completo pero el Policy Engine deniega toda tool con `side_effects != read` y registra lo que *habría* hecho. Es la funcionalidad que permite probar un agente de contacto con clientes sin escribir a nadie, y debe existir desde el principio, no añadirse tras el primer incidente.
-

23. OPTIMIZACIÓN DE COSTE

Ordenado por impacto real, de mayor a menor. Los tres primeros valen más que todos los demás juntos.

1. Usar el nivel de autonomía más bajo que funcione. Un L1 cuesta una llamada; un L3 cuesta entre diez y treinta. Gran parte de lo que se implementa como "agente autónomo" es en realidad una clasificación seguida de una plantilla. Ahorro: 10–30×.

2. Presupuestos duros por run. No es solo protección: cambia el comportamiento del diseño, porque obliga a que cada agente sea eficiente para caber en su presupuesto. Ahorro: elimina la cola de gastos catastróficos, que es donde se va el dinero de verdad.

3. Podar el contexto de forma agresiva. El coste es superlineal en la longitud del contexto a lo largo de un bucle: si en cada iteración se reenvía todo el historial, el gasto crece cuadráticamente con los pasos. Medidas: resumir observaciones antiguas, truncar resultados de tools (con el texto completo guardado en el event log y recuperable bajo demanda), no reenviar los schemas de tools que el agente ya no puede usar en esta fase. Ahorro: 3–10× en agentes iterativos.

4. Caché de resultados de tools deterministas. Clave = `hash(tool_id, args)`. Con TTL por tool. Búsquedas web y peticiones HTTP repetidas son el caso típico. Ahorro: variable, a menudo alto.

5. Caché semántica de llamadas al LLM. Solo para prompts idénticos (hash exacto) en V1; la caché semántica difusa introduce errores sutiles y no compensa aún.

6. Modelo pequeño para tareas mecánicas. Extracción, clasificación y formateo van a `cheap` por norma. Solo la planificación y el razonamiento sobre incertidumbre justifican `heavy`.

7. Salida estructurada en vez de prosa. Pedir JSON acotado reduce tokens de salida —que son los caros— y elimina el parseo frágil.

8. Programar por eventos, no por reloj. Un agente que corre cada hora "por si acaso" gasta 24 veces al día para no encontrar nada. Preferir disparadores sobre condiciones del Entity Store.

9. Coste de GPU: apagar es la optimización. Un script `scripts/gpu_up.sh / gpu_down.sh` y una regla personal de no dejar instancias encendidas. La causa número uno de gasto en Vast.ai es una instancia olvidada durante una noche.

Instrumentación mínima obligatoria: coste por run, por agente, por versión y por día, visible en el dashboard, con alerta al superar un umbral diario. No se puede optimizar lo que no se mide, y este sistema mide el coste desde el primer run porque `LLMInterface` lo registra siempre.

24. ARQUITECTURA MÍNIMA VIABLE (V1)

24.1 Qué entra

```
Python 3.11 · FastAPI · SQLite(WAL) · Pydantic · APScheduler · httpx
Docker Compose (api + worker)
LLM: provider mock + un provider OpenAI-compatible
UI: formularios generados desde JSON Schema (etapa B1)
6 tools: search.web, http.get, llm.extract, db.upsert, memory.*, knowledge.search
Niveles de autonomía L0, L1, L2, L3
1 agente vertical real: Demand Hunter
```

24.2 Qué queda explícitamente fuera de la V1, y por qué

Componente	Motivo de exclusión
Kubernetes, microservicios	Un problema que no tienes. Coste enorme, beneficio cero a esta escala.
Kafka / RabbitMQ / colas distribuidas	El scheduler + una tabla <code>runs</code> con estado <code>queued</code> resuelve lo mismo con 50 líneas.
Base vectorial dedicada	SQLite basta hasta cientos de miles de vectores.
vLLM + Qwen local	Diferido a M12 por economía (§18.3). La abstracción ya lo contempla.
Canvas visual de workflows	Diferido a B2 (§15.1). El vocabulario de nodos aún no está probado.
Multi-tenancy, roles, facturación	Un solo operador. Añadir después es viable; anticipar es fatal.
<code>shell.exec</code> , tools subidas por el usuario	Riesgo desproporcionado frente al beneficio actual.
Nivel de autonomía L4	Sin evaluación madura, un L4 es un generador de gasto impredecible.
Fine-tuning / RLHF	No hay datos, no hay señal, no hay necesidad.
Agentes multi-rol conversando entre sí	Composición determinista (<code>agent.invoke</code> + grafo) es más barata, más fiable y más depurable.

24.3 Criterio de "V1 terminada"

La V1 está terminada cuando esta secuencia se ejecuta de principio a fin sin intervención de un programador:

```
crear agente en la UI → guardar (versión 1) → programar a las 7:00
→ se ejecuta solo → observa (search.web) → razona → decide
→ el Policy Engine autoriza → ejecuta tools → escribe en domain.db
→ actualiza estado → evalúa criterio de éxito → termina y persiste
→ la traza es inspeccionable paso a paso
→ duplicar el agente, cambiar objetivo y política en la UI, guardar
→ el nuevo agente corre – SIN un solo commit en core/
```

PARTE III — IMPLEMENTACIÓN

25. PROTOCOLO DE SESIÓN DE TRABAJO

Toda sesión de construcción —con o sin GPU— sigue el mismo ciclo, y ninguna sesión termina sin commit. La regla que gobierna esto:

Ninguna sesión puede terminar con trabajo valioso viviendo solo en una máquina efímera.

1. ANALIZAR releer el hito: qué entra, qué no, cuál es el criterio de aceptación
2. DISEÑAR escribir el contrato (firmas, schema, tabla) ANTES del código
3. ESQUELETO ficheros, firmas, tipos, tests que fallan
4. IMPLEMENTAR la unidad más pequeña que hace pasar un test
5. PROBAR tests con el provider mock, sin coste
6. COMMIT commit pequeño y descriptivo · push inmediato
7. PERSISTIR exportar definiciones, snapshot de DB si cambió el schema
8. REGISTRAR una línea en docs/DECISIONS si se tomó una decisión no obvia
9. SIGUIENTE componente

Reglas de higiene:

- **Push cada 30-45 minutos.** No al final. Al final la instancia se cae.
- **Un hito = una rama = un PR** (aunque seas el único que lo revisa). El PR es el sitio donde revisas tu propio diff con ojos frescos.
- **Nada se marca terminado sin un test que lo demuestre** ejecutándose contra el provider `mock`.
- **Si vas a alquilar GPU, prepara el trabajo antes de encenderla.** El diseño y el esqueleto no necesitan GPU; escribirlos con el contador corriendo es tirar dinero.

26. HOJA DE RUTA POR HITOS

Cada hito es independientemente implementable, testeable y versionable. La columna GPU responde a "¿hace falta GPU para este hito?".

#	Hito	Entregable verificable	GPU	Esfuerzo
M0	Esqueleto y andamiaje	Repo, estructura, Docker, CI, <code>/health</code> responde, tests corren	No	1 sesión
M1	Agent Definition	JSON Schema v1, validador, <code>control.db</code> con <code>agents/agent_versions</code> , CLI <code>validate/import/export</code> , hash de contenido, round-trip idempotente	No	2 sesiones
M2	LLM Interface	Contrato <code>complete</code> , provider <code>mock</code> , provider OpenAI-compatible, contabilidad de tokens y coste, timeouts, cadena de degradación	No	1 sesión
M3	Tools + Policy	<code>ToolSpec</code> , registro, broker con validación de I/O y timeout, Policy Engine con <code>authorize</code> , presupuestos, 3 tools (<code>http.get</code> , <code>search.web</code> , <code>llm.extract</code>)	No	2 sesiones
M4	Event Log + Estado	Tabla <code>events</code> , emisor, <code>run_state</code> con control optimista, redacción de secretos, CLI <code>aap trace <run_id></code>	No	1 sesión

#	Hito	Entregable verificable	GPU	Esfuerzo
M5	Runtime L0/L1	Ejecutor, máquina de estados, presupuesto, terminación tipada. Primer agente ejecutándose de punta a punta con el provider mock	No	2 sesiones
M6	Runtime L2/L3	Planificador, bucle iterativo, replanificación, poda de contexto, <code>max_iterations</code>	Opcional	2 sesiones
M7	Demand Hunter v1	El primer agente vertical real: <code>domain.db</code> , deduplicación, tools de dominio, señales reales escritas en la base	Opcional	3 sesiones
M8	API + Scheduler + Persistencia	FastAPI completa, worker separado, cron, <code>dry_run</code> , backup/restore probado en máquina limpia	No	2 sesiones
M9	Factory y versionado	Duplicar, diff, promocionar, revertir, plantillas. Prueba de fuego \$7: agente nuevo sin commits en core/	No	2 sesiones
M10	UI B1	Formularios generados desde el schema, inspector de runs, cola de aprobaciones, dashboard de coste	No	4 sesiones
M11	Evaluación	Métricas mecánicas, eval sets, comparación de versiones, tabla <code>outcomes</code> + webhook de registro	No	2 sesiones
M12	Inferencia local	vLLM en Vast.ai, <code>gpu_up/gpu_down</code> , provider Qwen, benchmark coste/calidad frente a API	Sí	2 sesiones
M13	Segundo y tercer agente	Proposal Agent y un L0 de ingesta. Valida la generalidad del núcleo con casos distintos	No	3 sesiones
M14	Bucle de mejora	Detección de patrones, propuestas de diff, revisión humana, promoción	No	2 sesiones
B2	Canvas visual	Grafo de workflow, composición de agentes	No	4 sesiones

Camino crítico real: M0 → M1 → M2 → M3 → M5 → M7. En ese punto tienes un agente autónomo real produciendo datos de negocio reales. Todo lo demás es amplificación. Si algo se retrasa, que no sea nada de esa cadena.

Dónde entra el primer valor económico: M7. Ese es el hito que debe llegar cuanto antes; los hitos M8-M11 existen para convertir ese valor puntual en un sistema, pero no deben precederlo.

27. LO QUE YA SE HA HECHO Y LO QUE FALTA

Bloque	Estado a 30-ago-2026	Siguiente acción
Visión y modelo mental	Hecho (este documento)	Releerlo antes de cada hito
Principios arquitectónicos	Hecho	Convertir P1-P10 en tests donde sea posible
Diseño de componentes y contratos	Hecho	Traducir a firmas en M1-M3
Schema de Agent Definition	Diseñado , no implementado	M1
Modelo de datos	Diseñado (14 tablas, 3 DB)	M1 (control) · M4 (runtime) · M7 (domain)
API	Especificada (28 endpoints)	M8
UI	Especificada (15 pantallas)	M10

Bloque	Estado a 30-ago-2026	Siguiente acción
Repositorio	Especificado	M0
Runtime	Nada	M5
Tools	Nada	M3
Evaluación	Diseñada en 3 capas	M11
Agentes verticales	Nada en esta plataforma	M7
Trabajo previo (Demand Hunter, dedup, ADB)	Existe fuera	Canibalizarlo como tools en M7, no como base del runtime

28. CÓMO ENTREGAR ESTO A UN AGENTE DE CODING

Este documento no es un prompt de implementación: es la especificación. Para cada hito, el encargo al agente de coding debe contener exactamente esto y nada más:

1. La sección del documento que corresponde al hito (no el documento entero: el exceso de contexto degrada la implementación).
2. El criterio de aceptación en forma de tests que deben pasar.
3. Las tres reglas de dependencia de §20.2.
4. La restricción: *"no añadas dependencias nuevas sin justificarlas; no crees abstracciones para casos que no existen todavía"*.
5. La orden explícita de terminar con commit y push.

Un antipatrón a evitar: pedirle "implementa la plataforma". Producirá 4.000 líneas plausibles, con abstracciones inventadas, sin tests, y con `core/` contaminado. Los hitos existen precisamente para acotar ese riesgo.

PARTE IV — RIESGOS Y CUESTIONES ABIERTAS

29. RIESGOS

Ordenados por probabilidad × impacto. Los tres primeros son los que realmente matan proyectos como este.

R1 — Construir la fábrica sin haber fabricado nunca un trabajador. (Alta / Fatal) El síntoma es reconocible: cuatro meses de infraestructura elegante, cero oportunidades detectadas, entusiasmo agotado. *Mitigación:* el hito M7 (un agente real produciendo datos reales) es la puerta que todo lo demás debe cruzar. Si a las diez sesiones de trabajo no existe un agente que haya escrito una fila útil en `domain.db`, hay que parar y replantear.

R2 — Fuga de generalidad hacia el núcleo. (Alta / Grave) Un `if` específico de un agente en el runtime. Luego otro. En tres meses, `core/` sabe qué es un lead y la plataforma ha dejado de ser una plataforma. *Mitigación:* test de imports en CI, revisión del diff de `core/` en cada PR, y la prueba de fuego de §7 ejecutada en M9 y repetida en M13.

R3 — Gasto descontrolado. (Media-alta / Grave) Un bucle L3 sin límite, una instancia GPU olvidada encendida, un agente reintentando 400 veces. *Mitigación:* presupuesto obligatorio en el schema (sin valor por defecto infinito), alerta diaria de coste, `dry_run` por defecto para agentes nuevos, y el hábito de `gpu_down.sh`.

R4 — Sobre-ingeniería temprana. (Alta / Media) Colas, buses, vectores, microservicios, un DSL de workflows completo. Cada pieza parece razonable en aislamiento; juntas producen un sistema que nadie termina. *Mitigación:* la lista de exclusiones de §24.2 es un contrato contigo mismo; cada incorporación exige un ADR que documente el problema medido que la justifica.

R5 — La UI se convierte en un producto paralelo. (Media / Grave) El frontend acaba consumiendo el 70 % del tiempo, y el runtime se estanca. *Mitigación:* generar formularios desde el schema en lugar de escribirlos a mano; aceptar una UI fea en B1; prohibir cualquier lógica de negocio en el frontend.

R6 — Evaluación que mide lo fácil en vez de lo importante. (Media / Grave) Dashboards llenos de tasas de completitud mientras nadie sabe si el sistema genera ingresos. *Mitigación:* la tabla `outcomes` y la atribución por versión existen desde M11; la métrica principal del dashboard es coste por outcome, no tasa de éxito.

R7 — Deriva entre la base de datos y Git. (Media / Media) Definiciones editadas en la UI que nunca se exportan; el `.db` se pierde y se pierden seis semanas de configuración. *Mitigación:* exportación automática en cada promoción de versión, más un test de round-trip en CI y un aviso en la UI si hay cambios sin exportar.

R8 — Bloqueos de plataformas externas. (Media / Media-grave) Automatizar WhatsApp o LinkedIn puede costar la cuenta, y en algunos casos infringe sus términos. Esto es un riesgo de negocio real, no un detalle técnico. *Mitigación:* limitar la tasa desde la política del agente, exigir aprobación humana para mensajería saliente, preferir APIs oficiales cuando existan (WhatsApp Business API frente a automatización de la app), y decidir conscientemente qué riesgo se acepta antes de construir esa capacidad, no después.

R9 — Corrupción de SQLite por concurrencia. (Baja / Grave) Dos procesos escribiendo, o una base sobre sistema de ficheros de red. *Mitigación:* WAL, un único escritor por fichero, prohibición explícita de NFS/SMB, backup diario verificado por restauración.

R10 — Deuda por migración del schema de definición. (Media / Media) El schema v1 se queda corto; hay quince agentes escritos contra él. *Mitigación:* `schema_version` desde el primer día y un directorio `migrations/` con migraciones de definición, no solo de base de datos. La primera migración conviene escribirla pronto —aunque sea trivial— para que el mecanismo exista y esté probado antes de necesitarlo.

30. PREGUNTAS ARQUITECTÓNICAS AÚN ABIERTAS

Son decisiones que **no** conviene tomar hoy, porque dependen de información que aún no tienes. Deben revisarse en los hitos que se indican.

1. **¿El grafo de workflow y el bucle del agente conviven en un solo modelo de ejecución, o son dos motores distintos?** El diseño actual dice: el grafo orquesta, el bucle vive dentro de los nodos `agent`. Es lo correcto para empezar, pero cuando alguien quiera un grafo con memoria compartida entre nodos, esa frontera se tensará. *Revisar en B2.*
2. **¿Cómo se comparte el estado entre agentes encadenados?** Hoy: por el Entity Store, que es explícito y auditable. La alternativa —un contexto compartido entre runs— es más cómoda y mucho más difícil de depurar. *Revisar en M13, con dos o tres cadenas reales funcionando.*
3. **¿La memoria de largo plazo debe ser por agente, por versión o compartida entre agentes?** Hoy: por agente, no heredada al duplicar. Un pool de memoria compartido entre agentes de un mismo dominio es tentador y potencialmente contaminante. *Revisar en M13.*
4. **¿Cuándo deja de bastar SQLite?** Hay un disparador escrito (§21.3). La pregunta real es si el Entity Store crecerá más rápido de lo previsto y necesitará Postgres antes que el resto. *Revisar cuando `domain.db` supere 5 GB o los runs concurrentes pasen de 20.*
5. **¿Debe el criterio de éxito poder ser evaluado por un LLM, o solo por expresiones deterministas?** Hoy: solo deterministas, porque un criterio evaluado por LLM es un juez con incentivo a aprobarse a sí mismo. Pero hay objetivos cualitativos legítimos ("la propuesta es persuasiva"). *Revisar en M11, con datos de la fiabilidad del juez-LLM.*
6. **¿Multi-tenancy alguna vez?** Si esto llega a ser un producto para terceros, cambia la seguridad, el aislamiento de tools y el modelo de datos. La decisión de *no* prepararlo ahora es deliberada y correcta, pero conviene saber que su coste posterior es de semanas, no de días. *Revisar solo si aparece un cliente real.*
7. **¿Los agentes deberían poder crear otros agentes?** Es la conclusión lógica de "fábrica de trabajadores": un meta-agente que fabrica agentes. Técnicamente ya sería posible en cuanto exista la API (un agente con una tool que llame a `POST /agents`). *La respuesta hoy es no*, y la razón es que sin evaluación madura sería un generador de agentes malos y no auditados. *Revisar después de M14.*
8. **¿Cuál es la unidad de negocio real: el agente o el resultado?** Si algún día esto se vende, ¿se vende "un agente" o "reuniones agendadas"? La respuesta cambia el producto, la UI y las métricas, aunque no cambia el núcleo. Vale la pena tenerla presente mientras se construye. *Revisar cuando haya outcomes reales que contar.*

31. GLOSARIO

Término	Significado en este sistema
Agent	Identidad lógica que agrupa versiones. No se ejecuta; se ejecutan sus versiones.

Término	Significado en este sistema
Agent Definition	Documento JSON declarativo que describe por completo un agente.
Agent Version	Definición congelada e inmutable, identificada por hash de contenido.
Run	Una ejecución concreta de una versión, con estado, eventos, coste y resultado.
Runtime	El intérprete que ejecuta versiones. No conoce ningún agente concreto.
Tool	Capacidad de percepción o acción, con schema de entrada y salida y permisos.
Policy Engine	Punto de paso obligatorio entre decisión y ejecución.
Budget	Límite duro de pasos, tokens, dinero y tiempo por run. Es una política.
Working Memory	Contexto efímero del run. No se persiste; se reconstruye del event log.
Run State	Estado tipado y persistido del run, declarado en <code>state_schema</code> .
Long-term Memory	Afirmaciones curadas, con procedencia y caducidad. Escritura explícita.
Knowledge	Documentos de solo lectura ligados a la versión.
Entity Store	Base de datos del negocio (<code>domain.db</code>). Sobrevive a todos los agentes.
Outcome	Consecuencia medible en el mundo real, atribuida a run y versión.
Capability	Clase de modelo requerida (<code>cheap / standard / heavy</code>). El agente nunca nombra un modelo.
Autonomy Level	L0-L4. Determina la forma del bucle de ejecución.
Factory	Capa de creación, clonado, versionado y promoción. No ejecuta nada.
Eval Set	Conjunto congelado de escenarios para comparar versiones.
ADR	Registro de decisión arquitectónica: un fichero corto por decisión no obvia.

32. CORRESPONDENCIA CON EL BRIEF ORIGINAL

Para verificar cobertura. Los puntos donde este documento se aparta del brief están marcados con △.

Brief	Aquí	Nota
1-3 Contexto, visión, DATA→OUTCOME	§1	Añadida la atribución diferida de outcomes
4-5 Stack y GPU efímera	§18	△ Se difiere vLLM/Qwen a M12 por economía
6 Minimum Autonomous Agent Platform	§24	
7 La gran separación	§3, §6, §7	
8-9 No-code y canvas visual	§15	△ Definición declarativa desde el día 1; canvas diferido a B2
10 Source of truth	§14	JSON canónico en SQLite + export YAML a Git
11 Duplicación	§16.3	△ La memoria no se hereda por defecto
12 Ciclo autónomo y niveles	§8	△ Se argumenta que el valor está en L0-L2
13-14 Model ≠ Agent, routing	§17	Router = tabla + cadena de degradación en V1
15 Tool system	§10	△ Catálogo recortado; <code>shell.exec</code> fuera de V1
16 Autoridad del modelo	§11	Chokepoint estructural, no convención

Brief	Aquí	Nota
17 Memoria vs estado vs eventos	§9	△ Se añade el Entity Store como sexta categoría
18 Event system	§9.4	△ Log de eventos, no bus. 11 tipos, no 12
19 Evaluation	§12	Tres capas; outcomes como capa reina
20 Self-improvement	§13	△ Solo propuestas; nada auto-promociona en V1-V2
21 Observability	§9.4, §12.2	△ Fusionada con el event log: un subsistema, no tres
22 Safety / policies	§11	
23 Visual configuration UX	§15.2	15 pantallas con separación normal/avanzado
24 Dos tipos de usuario	§15.3	
25 Agent Factory	§16	
26 Cloud vs Local	§18	△ Tres capas, no dos: falta el plano de control 24/7
27 Docker	§19	
28 Repository	§20	△ Se rechazan 6 de los 16 directorios propuestos
29 API	§22	Control plane + backend de UI; el runtime no se expone
30 Database	§21	△ Tres ficheros SQLite, no uno
31 Versioning	§16.4	DB para ejecutar, Git para revisar
32 Reconstrucción	§19.3	Requisito con prueba real en M8
33 Fabricación con tiempo limitado	§25	
34 Reutilización	§4 (P8), §20.2	
35 Minimum viable architecture	§24	
36-43 Formato del entregable	Todo el documento	

33. CIERRE: LAS CINCO COSAS QUE NO HAY QUE ROMPER

Si dentro de seis meses el proyecto se ha desviado, casi con seguridad será porque una de estas cinco cedió. Merecen estar en la pared:

1. **La Definición es el agente.** El código no sabe nada de agentes concretos.
2. **El modelo propone; el runtime dispone.** Un solo chokepoint, sin excepciones ni banderas de depuración.
3. **Todo run tiene presupuesto y toda ejecución deja traza.**
4. **El estado y el dominio sobreviven a la GPU, al contenedor y al agente.**
5. **Ningún componente entra sin un problema medido que lo justifique.**